

Parallel and Distributed Computing

Dr. Shajulin Benedict
shajulin@iiitkottayam.ac.in

Objectives

- The objectives of the course are
 - to introduce the concept and the basics of parallel and distributed computing,
 - to provide knowledge on various available parallel programming models
 - to enable awareness on the utilization of programming models on various scientific applications

Pre-requisites

- The participants should have prior knowledge on the following topics/courses:
 - Computer Networks
 - Fundamentals of Programming
 - Computer Organization

Syllabus

- Architectures – Multi-core and Many-core architectures, Accelerators (SIMD units -Vectorization, GPUs), Goals of parallel systems.
-
- Applications – Scientific applications, Characteristics, requirements, regular grid applications, irregular applications, data dependence, parallelization process.
-
- Parallel Programming on Shared Memory – OpenMP (C++ languages), Execution Model, Shared and private data, Directives, Barriers, Sections, Run-Time library functions, scheduling strategies, Scalability study, OpenMP for accelerator programming.
-
- Parallel Programming on Distributed Memory – MPI, Collective operations, Non-Blocking, Collectives, Process topologies, Parallel I/O, Single sided communications.
-
- Performance Tools – Concepts, Event-Model execution, profiling, tracing, types of profiling, profiling tools – Scalasca, Score-P, MPI-P, EnergyAnalyzer, Tracing tools, Autotuning – Periscope Tuning Framework.

Glossary – Terminology

- <https://www.cise.ufl.edu/research/ParallelPatterns/glossary.htm>

History of Parallelism

- Speed is quite important for any programmers.
- History
 - 1642 – Blaise Pascal's Calculating machine
 - 1812-1832 Charles Babbage's difference engine
 - 1853-1871 Analytical engine
 - 1930-1940 Electro mechanical computers
 - 1940- First electronic computer
 - 1950s - Compact systems
 - 1986- till date - Parallel and distributed systems

History of Parallelism

- Drawbacks of uniprocessor systems...
- In a single processor system there will never be more than one running process ...
 - Slow performance...
 - Task 1 → Task 2 → Task 3 →
 - Load is high in single core or processor
 - Leading to failures

History of parallelism: Moore's law

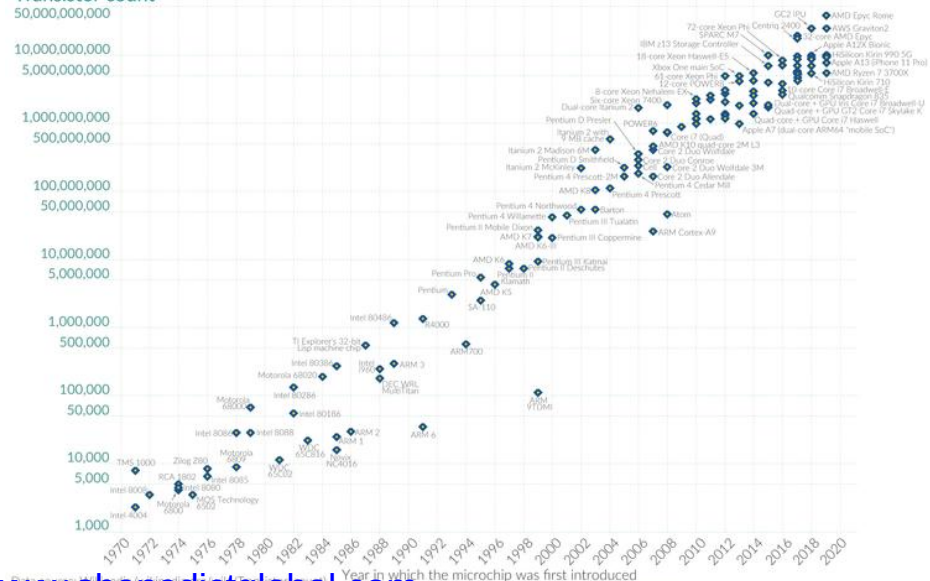
- Moore's law of Gordon Moore (Intel Co-Founder) observes that the number of transistors in a dense integrated circuit (IC) doubles about every two years.
- i.e., it would double a processor's performance every couple of years.
- i.e., the size of processors needs to be minimized.

Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World
in Data

Transistor count

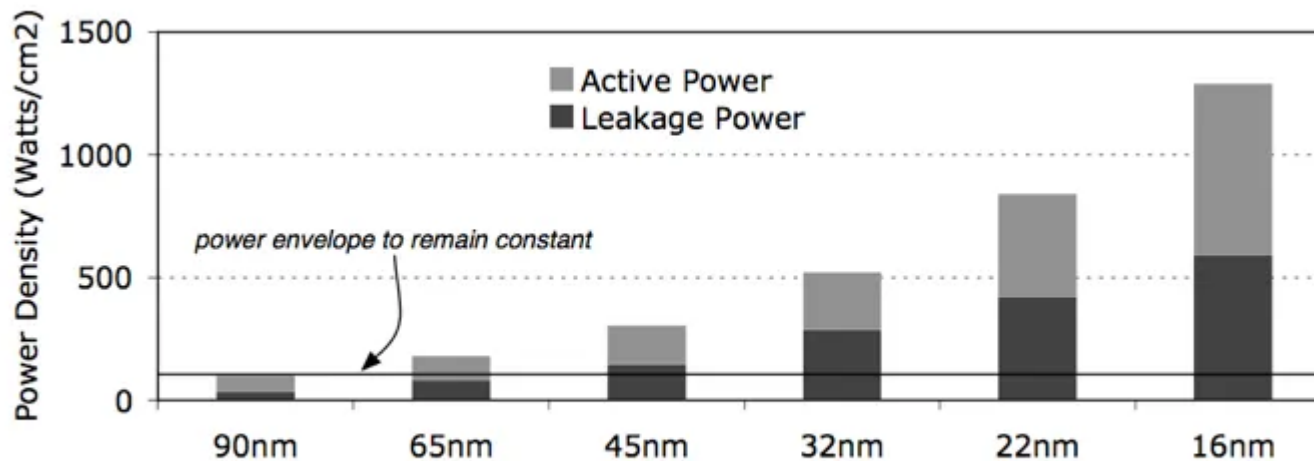


www.sbenedictglobal.com

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

History of parallelism: Moore's law fails...

- The CPU clock frequency did not increase as expected in the processor industry.
- Why?
 - Heat and Power



Source: Shekhar Borkar (Intel)

- Heat dissipation is too high.
- Solution: Bring newer transistor technologies.

Need for parallel computers

- It is a solution to solve Moore's law (to bring better performance every two years).
- Also, the requirement of software developers considering real-world scenario is to solve complex parallel problems.
- In short, parallelism is the future of computing.

Software vs. Hardware advances

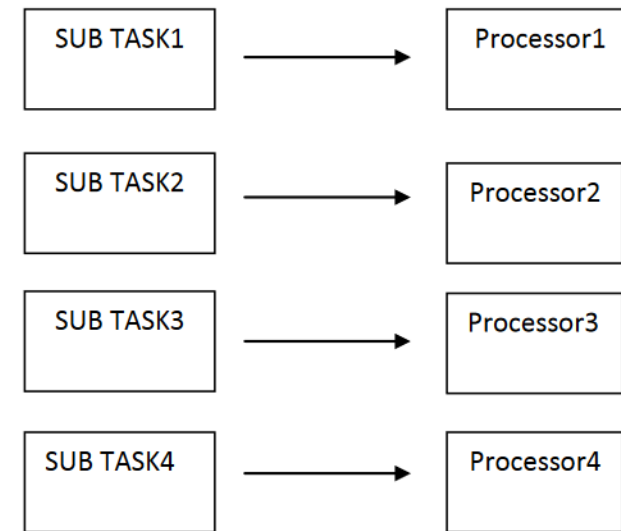
- Software and hardware mutually advances each other.
- They are tightly related each other.
- When software improves, hardware limitations are reached.
- When hardware improves, software limitations are found.
- For e.g., introducing multiprocessors, invited newer programming models such as MPI, OpenMP, and so forth.

Different Computing – Buzz...

- Serial computing
- Parallel computing
- Distributed computing
- Cloud Computing
- Grid computing
- Mobile computing
- Green computing
- Wearable computing
- Supercomputing
- Client-server computing
- Personal computing
- Serverless computing
- Embedded computing
- Real-time computing
- Social computing
- Edge computing
- Fog computing
- HPC
- Quantum computing

What is Parallel Computing?

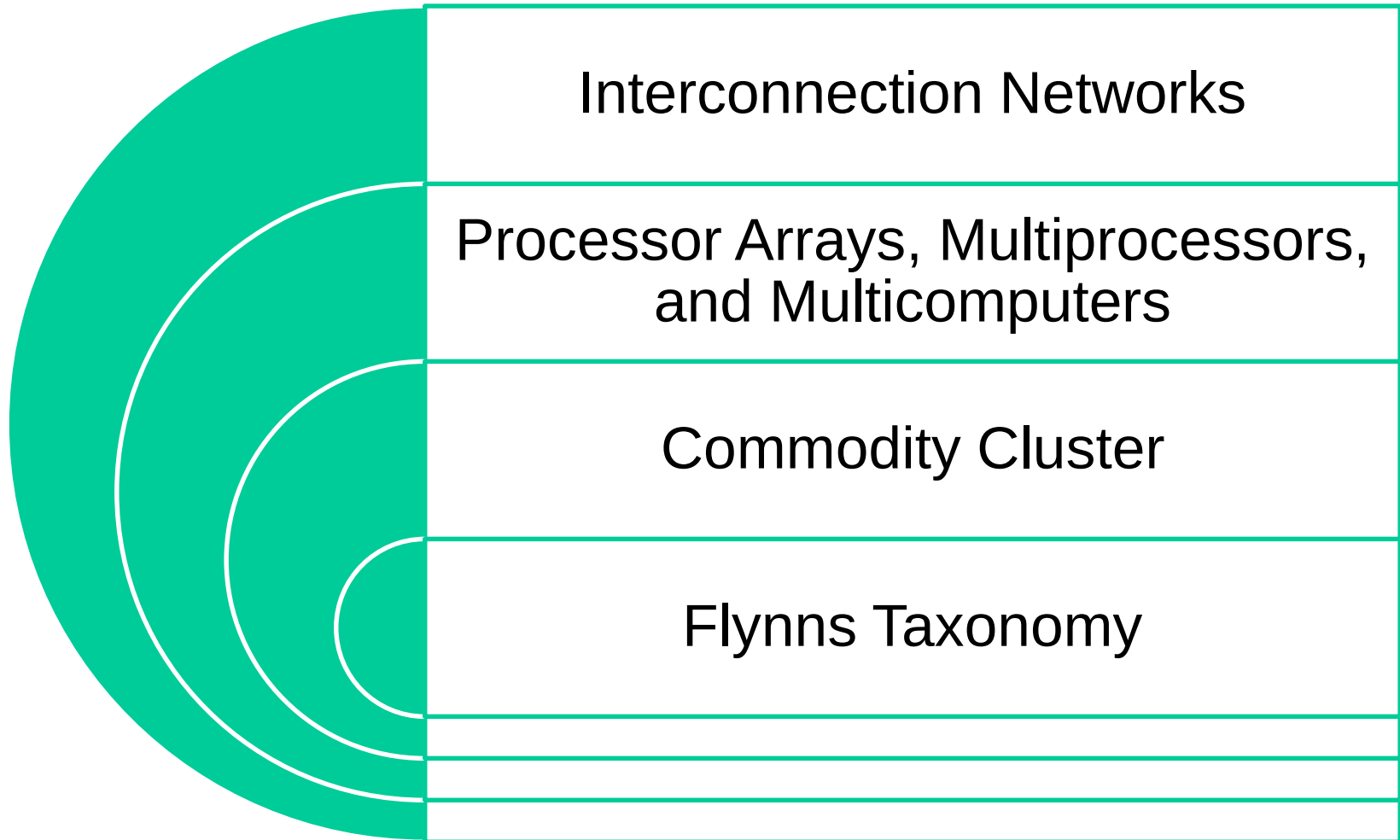
- Parallel processing is represented as follows:
- Parallel computing ?
 - Parallel computing is the use of **parallel computer** to reduce the time needed to solve a single computational problem.
- What is parallel computer?
 - It is a multiple-processor computer system supporting **parallel programming**.
- What is parallel programming?
 - It is a programming in a language that allows us to explicitly indicate how different portions of the computation may be executed **concurrently** on different processors



Concurrency – How?

- Seeking possibilities
 - Data dependence?
 - Functional dependence?
 - Can we apply pipelining?
 - Eg. Computing partial sums in a FOR loop
 - $P0 \leftarrow a0$
 - $P1 \leftarrow a0 + a1$
 - $P2 \leftarrow a0 + a1 + a2$
 - $P3 \leftarrow a0 + a1 + a2 + a3$
 - Concepts such as bypassing (Forwarding) or bubbling (delaying)
 - Any other approaches?
 - Extend compilers...
 - Extending programming languages (serial to parallel)
- Creating a new parallel programming language

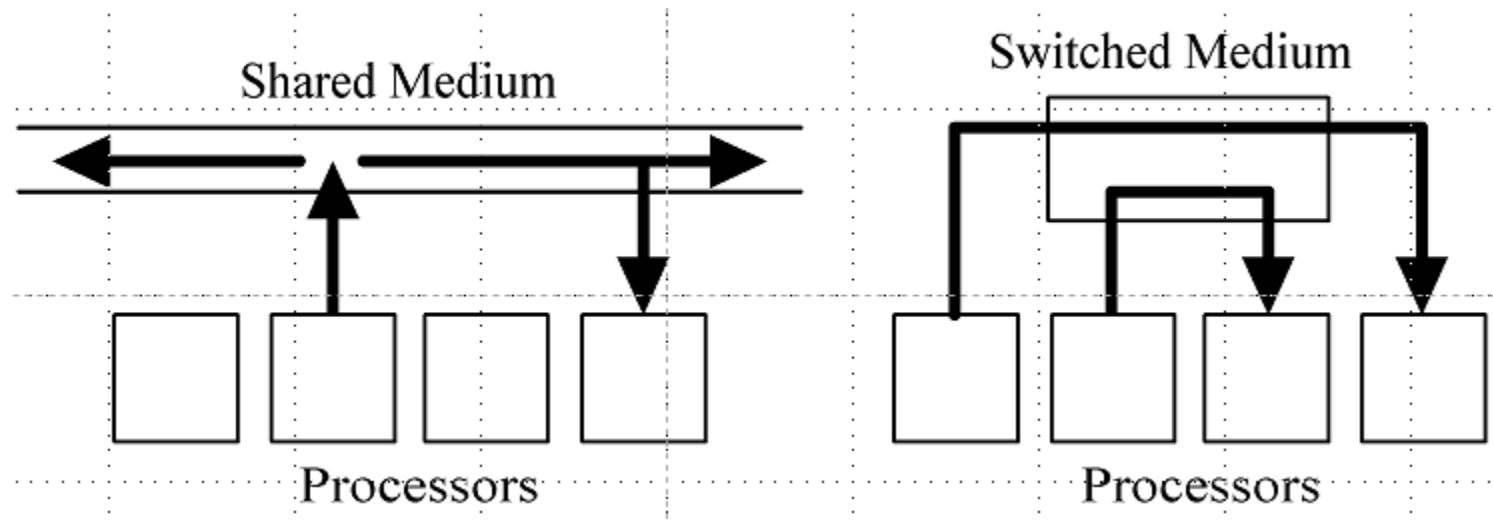
Parallel Architectures – To Discuss



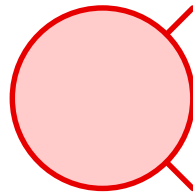
Interconnection Networks

- Uses of interconnection networks
 - Connect processors to shared memory
 - Connect processors to each other
- Interconnection Medium -- Types
 - Shared medium
 - Switched medium
- Different interconnection networks are utilized in different parallel machines.
- The interconnection network's properties influence the type of algorithm used for various machines as it affects how data is routed.

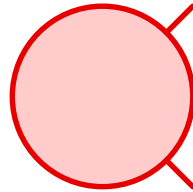
Shared vs. Switched Networks



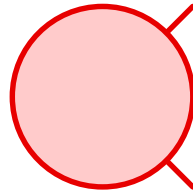
Shared Medium



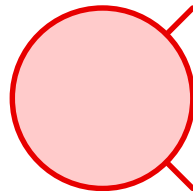
Shared medium allows one message at a time



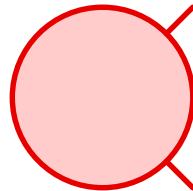
Messages are broadcast



Each processor “listens” to every message

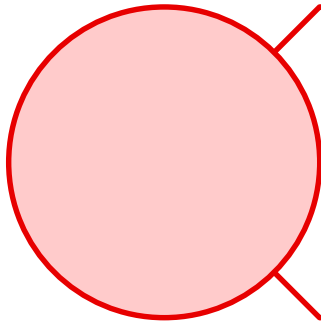


Before sending a message, a processor “listen” until medium is unused

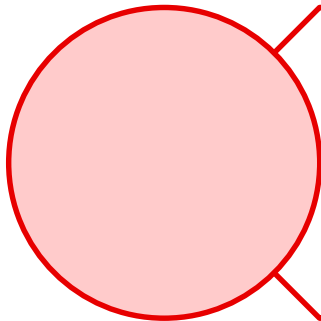


Collisions require resending of messages

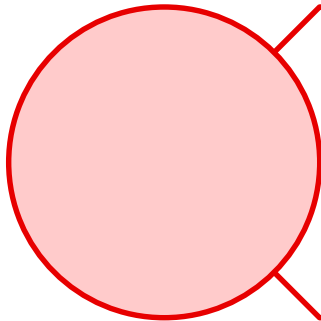
Switched medium



Supports point-to-point messages
between pairs of processors



Each processor is connected to
one switch



Advantages over shared media

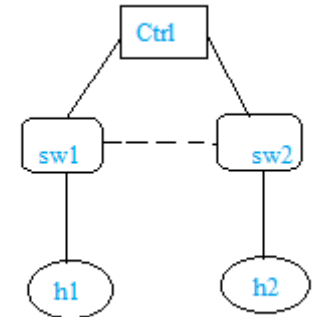
- Allows multiple messages to be sent simultaneously
- Allows scaling of the network to accommodate the increase in processors

Topologies – in Switched Networks

- 2 Topologies: Direct and Indirect topologies

- Direct Topologies:

- Ratio of switch nodes to processor nodes is 1:1
- Every switch node is connected to
 - 1 processor node
 - Or, at least 1 other switch node
 - Eg. Mesh networks



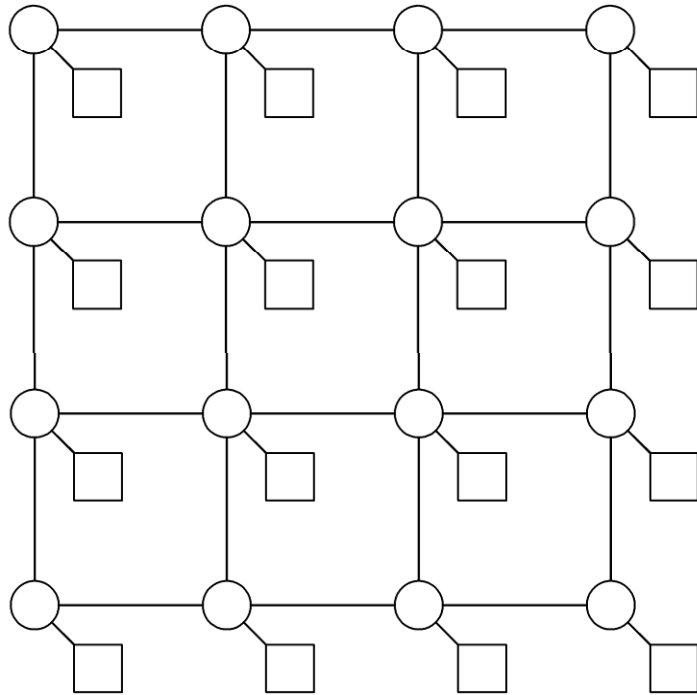
- In-direct Topologies:

- Ratio of switch nodes to processor nodes is greater than 1:1
- Some switches simply connect to other switches.

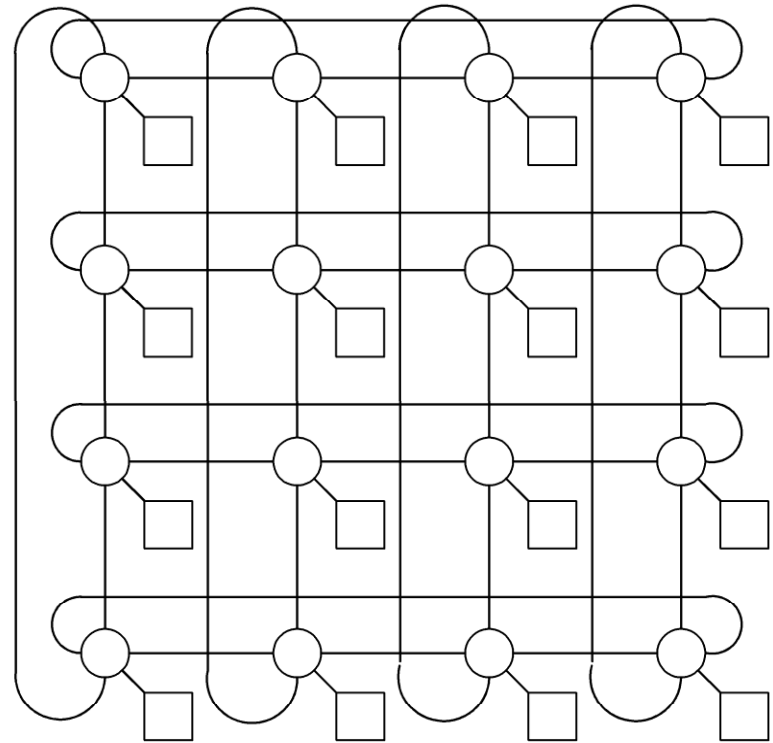
- Topologies – to understand

- 2D Mesh; Binary Tree; Hypertree; Butterfly; Hypercube; and Shuffle-exchange

2D Mesh network examples



(a)



(b)

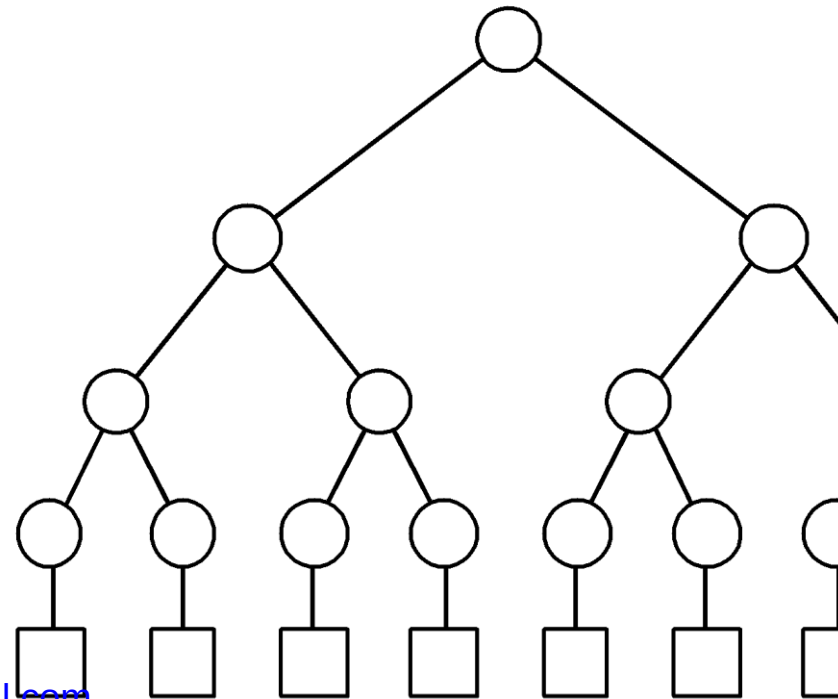
- Circles represent switches and squares represent processors...

2D Mesh Network

- Direct topology
- Switches are arranged into a 2-D lattice or grid
- Communication is allowed only between neighboring switches.
- In some cases, there is a possibility of wraparound connections (see figure b).

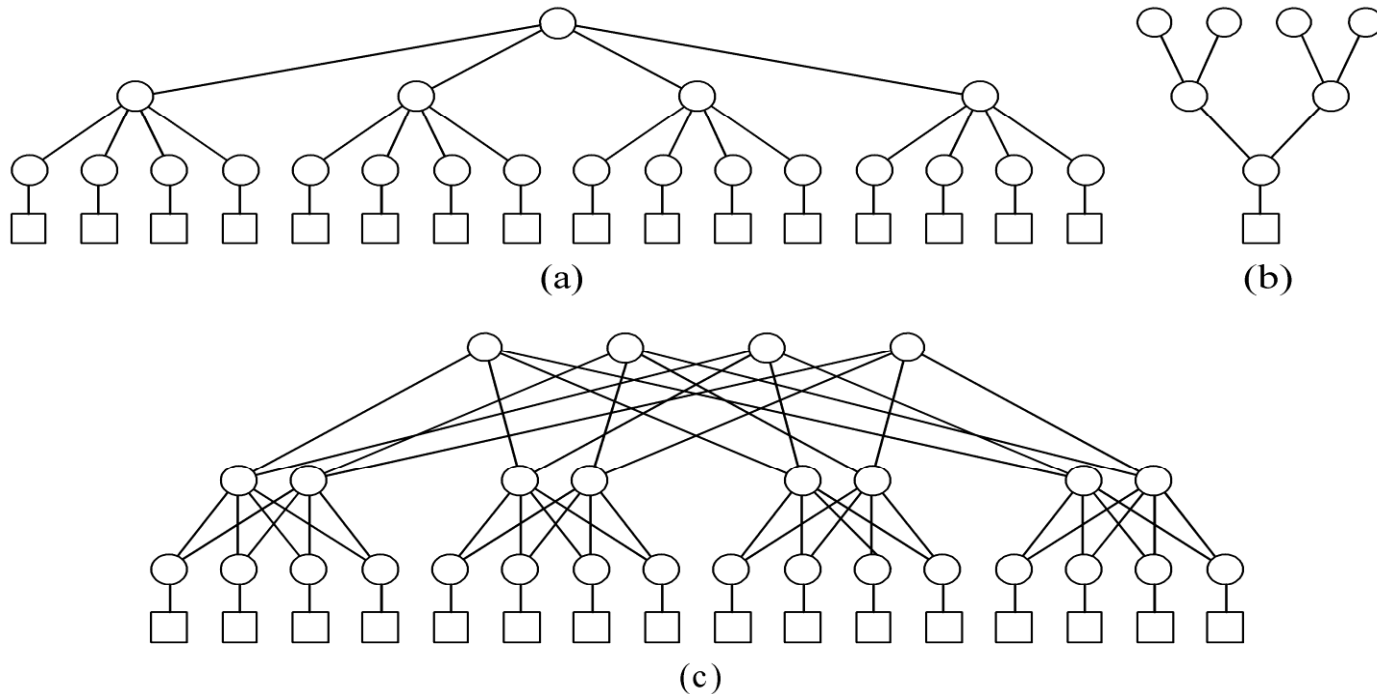
Binary Tree Network

- It is an indirect topology.
- Each processor node is connected to a leaf of the binary tree.
- Less efficient in terms of Degree of Nodes.



Hypertree Network

- Indirect topology.
- Better than binary tree topology.
- Different patterns of connections include:



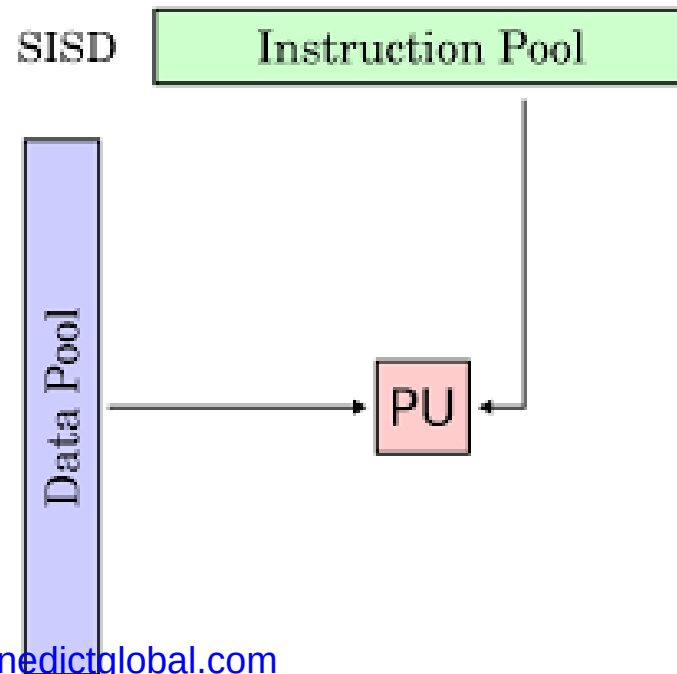
Parallel Computers – Classes

- Flynn's taxonomy is the classification scheme for parallel computers.
- Flynn's taxonomy is based on
 - Instruction stream
 - Data stream
- Single Instruction
 - Single/Common Control unit

		Instruction Streams	
		one	many
Data Streams	one	SISD traditional von Neumann single CPU computer	MISD May be pipelined Computers
	many	SIMD Vector processors fine grained data Parallel computers	MIMD Multi computers Multiprocessors

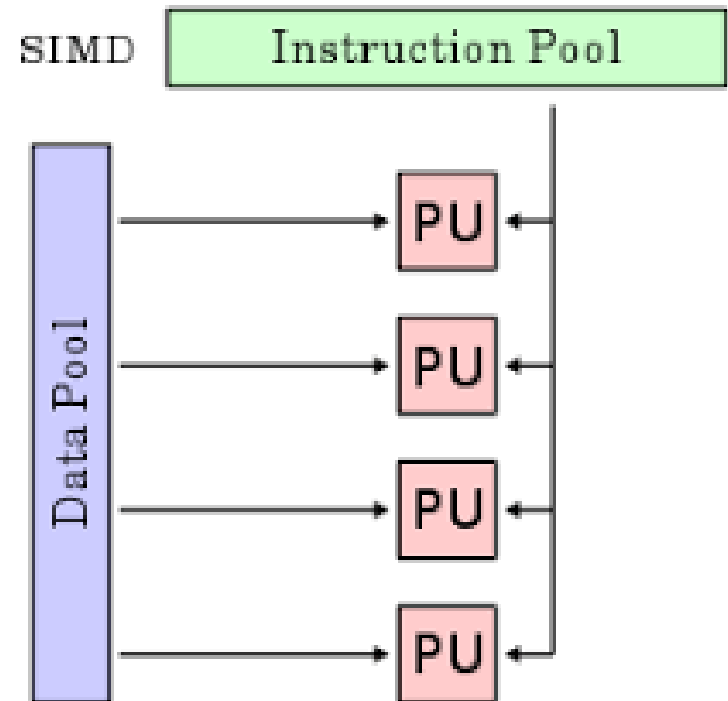
SISD

- It represents the organization of a single computer containing a control unit, processor unit and a memory unit.
- Instructions are executed sequentially.
- VonNeumann Model.
- Not so useful...



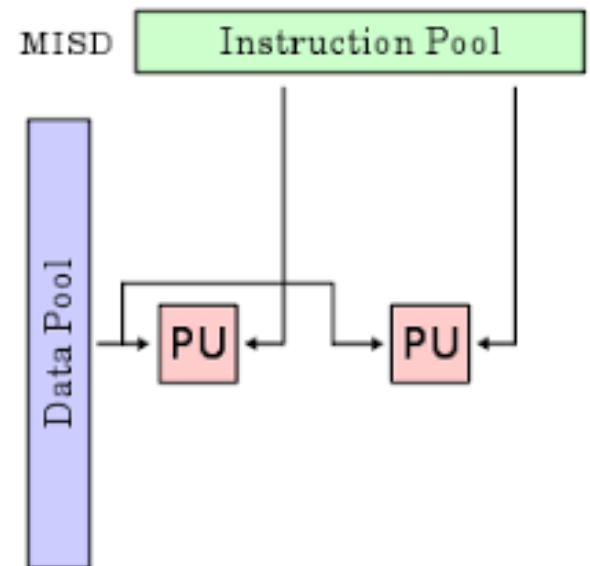
SIMD

- It represents an organization that includes multiple processing units under the control of a **common control unit**.
- All processors receive the **same instruction from control unit** but operate on different parts of the data.
- Why SIMD evolved?
- Eg. Vector...



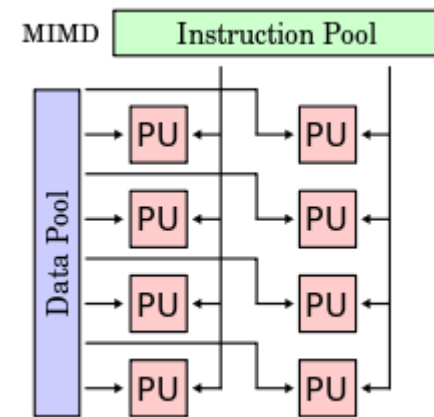
MISD

- It consists of a single computer containing multiple processors connected with **multiple control units** and a common memory unit.
- It is capable of processing several instructions over single data stream simultaneously.
- MISD structure is only of theoretical interest since **no practical system** has been constructed using this organization.



MIMD

- It represents the organization which is capable of processing several programs at same time.
- It is the organization of a single computer containing **multiple processors** connected with **multiple control units** and a shared memory unit.
- The shared memory unit contains multiple modules to communicate with all processors simultaneously.
- **Multiprocessors and multicomputer** are the examples of MIMD. It fulfills the demand of large scale computations.

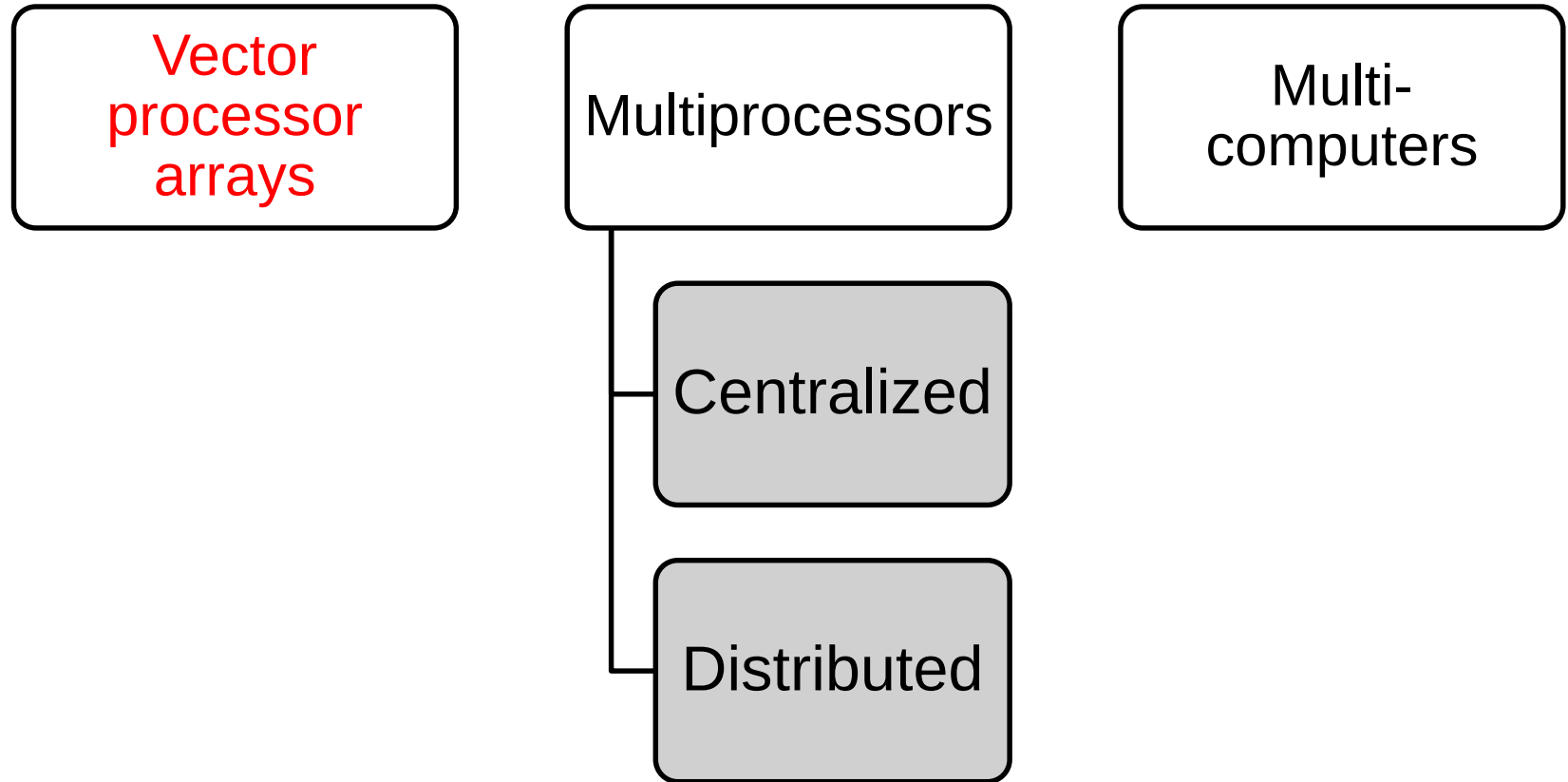


Flynns Taxonomy

FLYNN'S TAXONOMY OF PARALLEL MACHINES



Parallel Architectures – 3 Popular ones



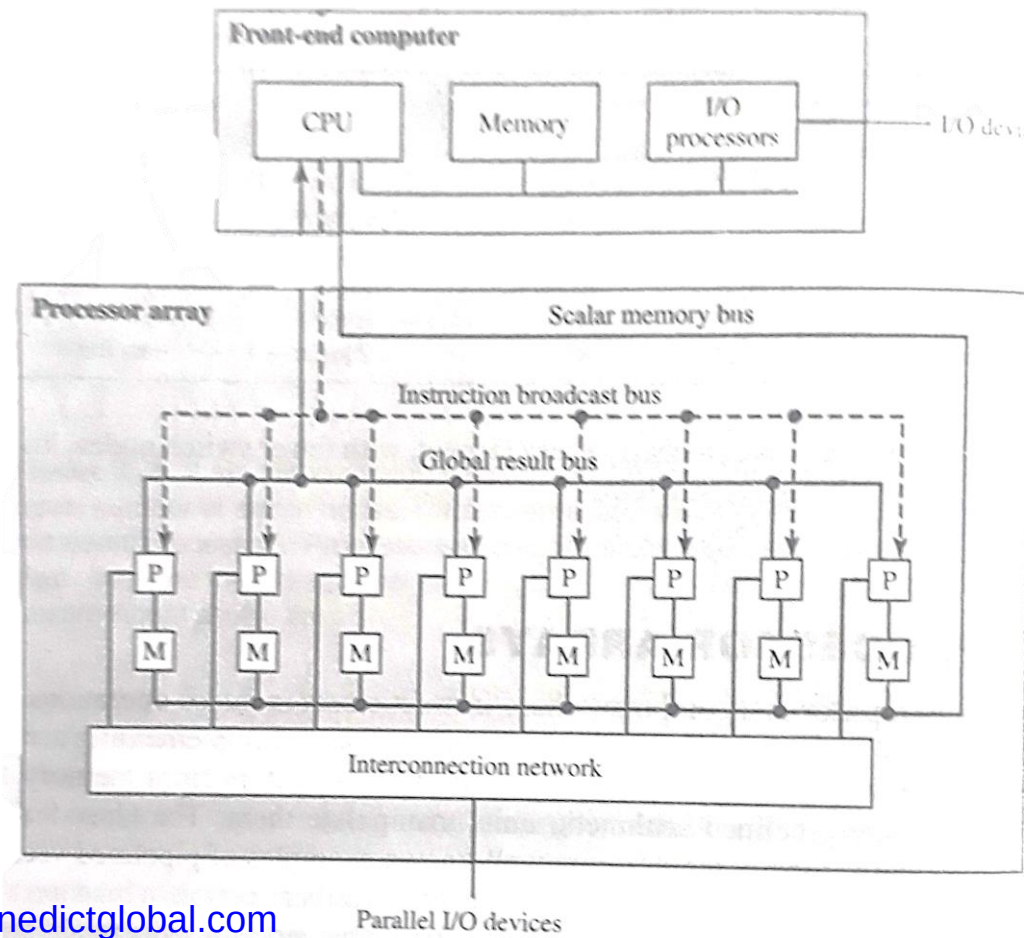
Vector Processor Arrays

- It is a vector computer implemented as a sequential computer connected to a set of identical processing elements
- (capable of simultaneously performing the same operation on different data).
- i.e., they perform computations on large arrays of data.
- **Motivation:** Many scientific applications are data parallel.
- The performance of applications could be heavily improved **for certain applications.**
 - Simulations involving multimedia processing,
 - Simulations such as structural engineering, aerodynamics, hydrodynamics, nuclear research, VLSI design, **and so forth.**

Vector processor Array - Architecture

- It is a collection of processing elements controlled by a frontend computer (uniprocessor).

-



Understanding Vector Processor Arrays

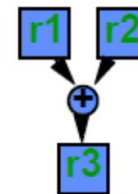
- A normal CPU

```
execute this loop 10 times
  read the next instruction and decode it
  fetch this number
  fetch that number
  add them
  put the result here
end loop
```

- A Vector processor array

```
read instruction and decode it
fetch these 10 numbers
fetch those 10 numbers
add them
put the results here
```

SCALAR
(1 operation)



add r3, r1, r2

VECTOR
(N operations)



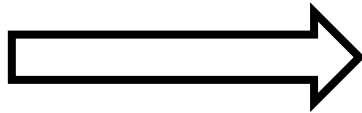
add.vv v3, v1, v2

NOTE: Fetching and Decoding
happen only once

https://en.wikipedia.org/wiki/Vector_processor

Vectorized Code

```
// C code  
for(i=0;i<16; i++)  
    b[i]+=a[i]
```



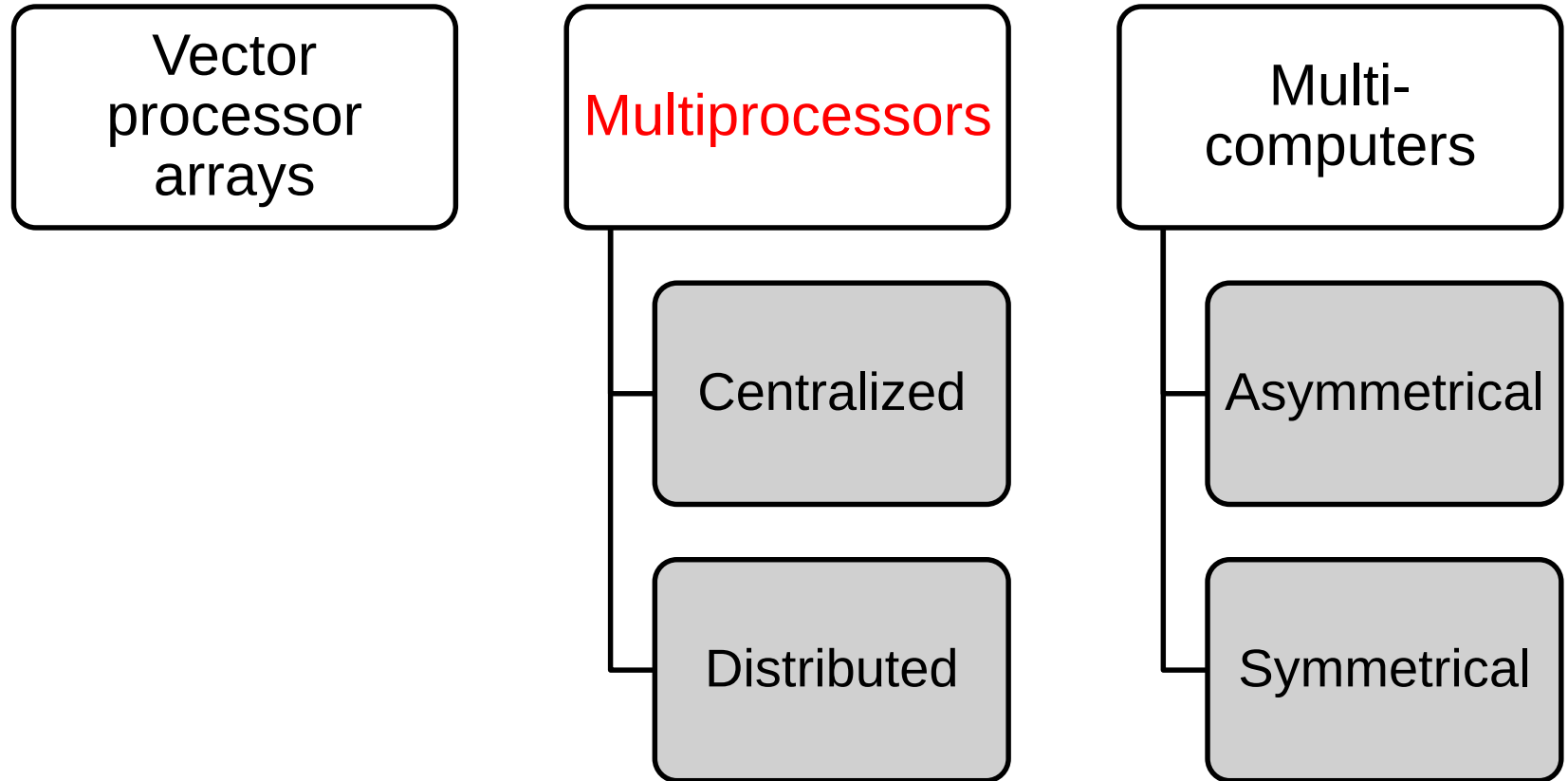
```
// Vectorized code  
set    vl,16  
vload  vr0,b  
vload  vr1,a  
vadd   vr0,vr0,vr1  
vstore vr0,b
```

Vload → vector load
Vr1 → vector register 1

Disadv. Of Vector Array Processors

- All applications do not fit to this model.
- Programs having conditions (Nested-if-then-else statements) produce poor efficiency.
- Increased code complexity needed to vectorize the data
- Manufacturing such processors require diligent VLSI coding – cost of such machines are high (high design cost).
- Processor arrays are sparsely applied in general-purpose computers.

Parallel Architectures – 3 Popular ones



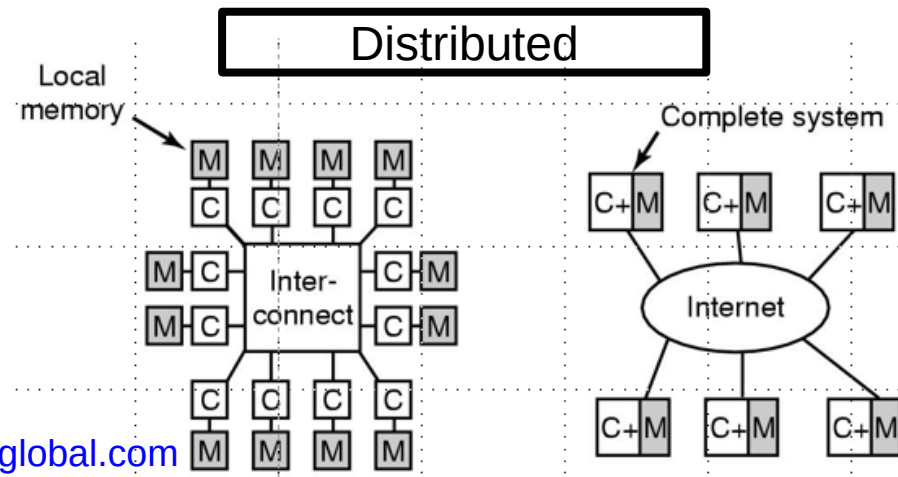
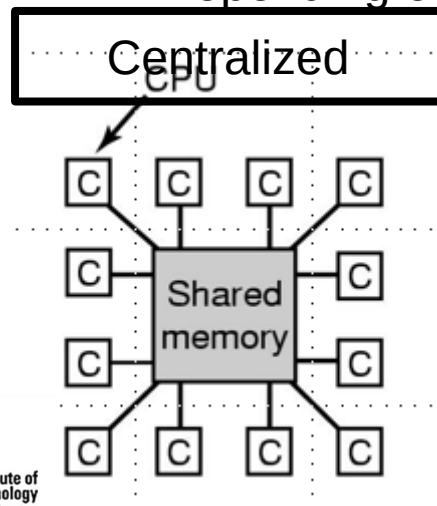
Multiprocessors

- What is a multiprocessor?

- As discussed, it is a computer system in which two or more CPUs share full access to a common RAM.

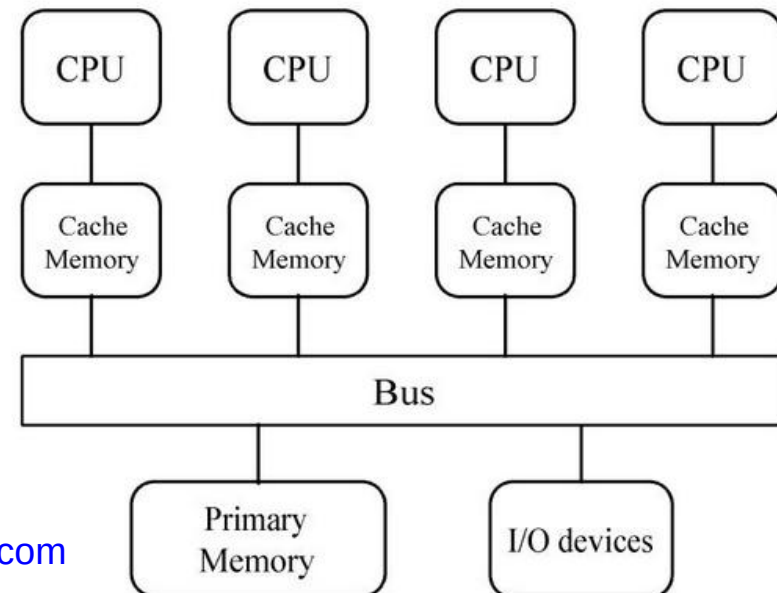
- Two fundamental types of multiprocessors

- Centralized multiprocessors
 - All primary memory is in one place.
- Distributed multiprocessors
 - All primary memory is distributed among the processors.
 - Depending on shared memory, distributed multiprocessors differ.



Centralized Multiprocessors

- A centralized multiprocessor is an extension of uniprocessors.
- i.e., additional CPUs are attached to the bus,
- All the processors **share the same primary memory**.
- Also named as ...
 - Uniform Memory Access multiprocessor (UMA)
 - Symmetric Multiprocessor (SMP)
- Private data
 - Data items used by one CPU
- Shared data
 - Data used by multiple CPUs

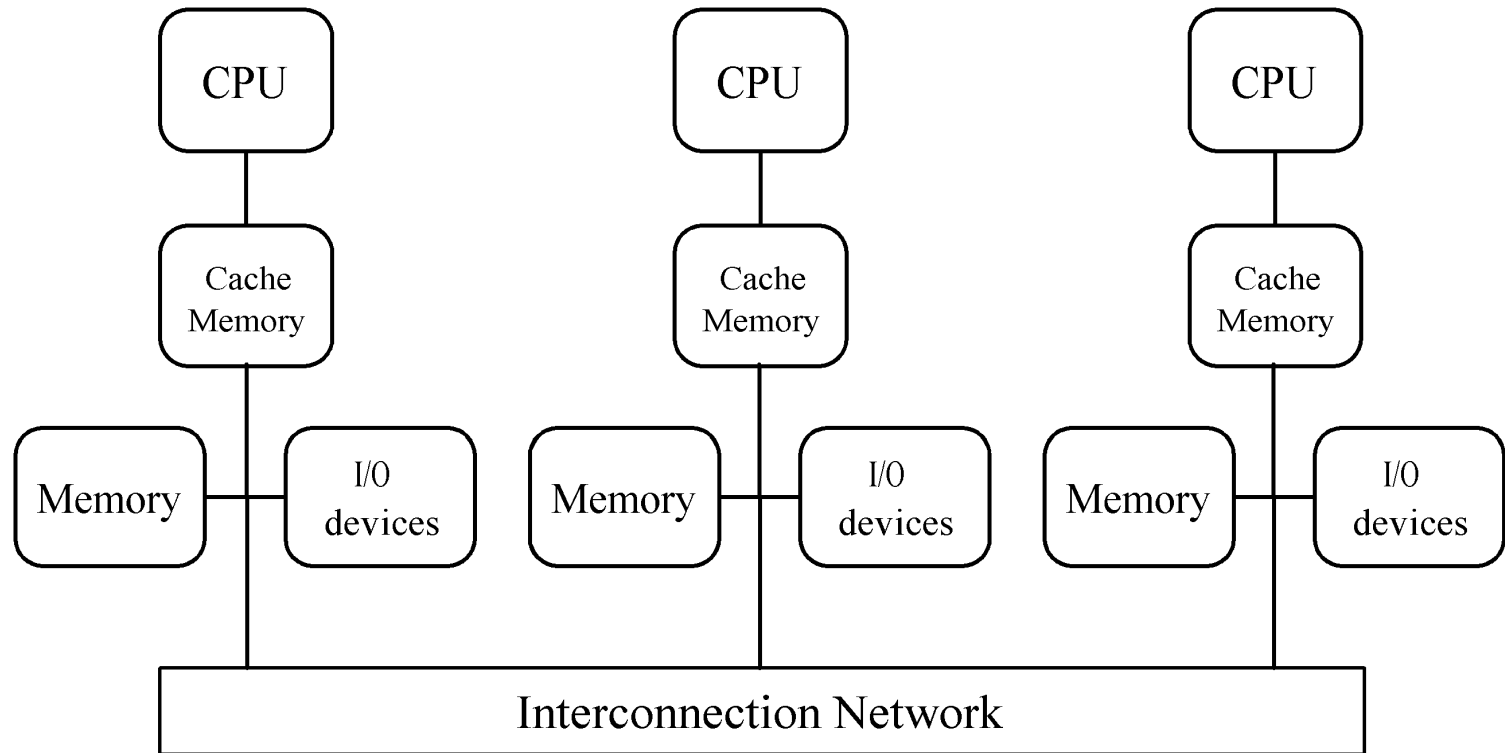


Distributed Multiprocessors

- Here, the **primary memory** is distributed among processors to form **one logical address space**.
- It increases the aggregate memory bandwidth and lowers the average memory access time.
- Allows greater number of processors.
- It is also called as *non-uniform memory access* (**NUMA**) multiprocessor
 - Local memory access time is fast
 - Non-local memory access time can vary...
 - Distributed memories have **one logical address space**.

Architecture

One Logical address space...



Multicomputers

- It is a multi-CPU computer architecture.
- It is a class of distributed multiprocessor **without** one logical shared memory.
- Such architecture has **disjoint local address spaces**.
 - i.e., there is no global address space as like in UMA or NUMA.
 - ie., Each processor has direct access to its own local memory.
- The processors interact with each other by passing messages. (MPI programming model).
- The switching network of such architectures should have
 - Low-latency
 - High bandwidth access (between processors)
- A good balance between the high speed processors and high speed networks is really a crucial task for multi-computers.

Centralized Multiprocessors -- Issues

Two problems arise while dealing with the shared data in the centralized multiprocessors

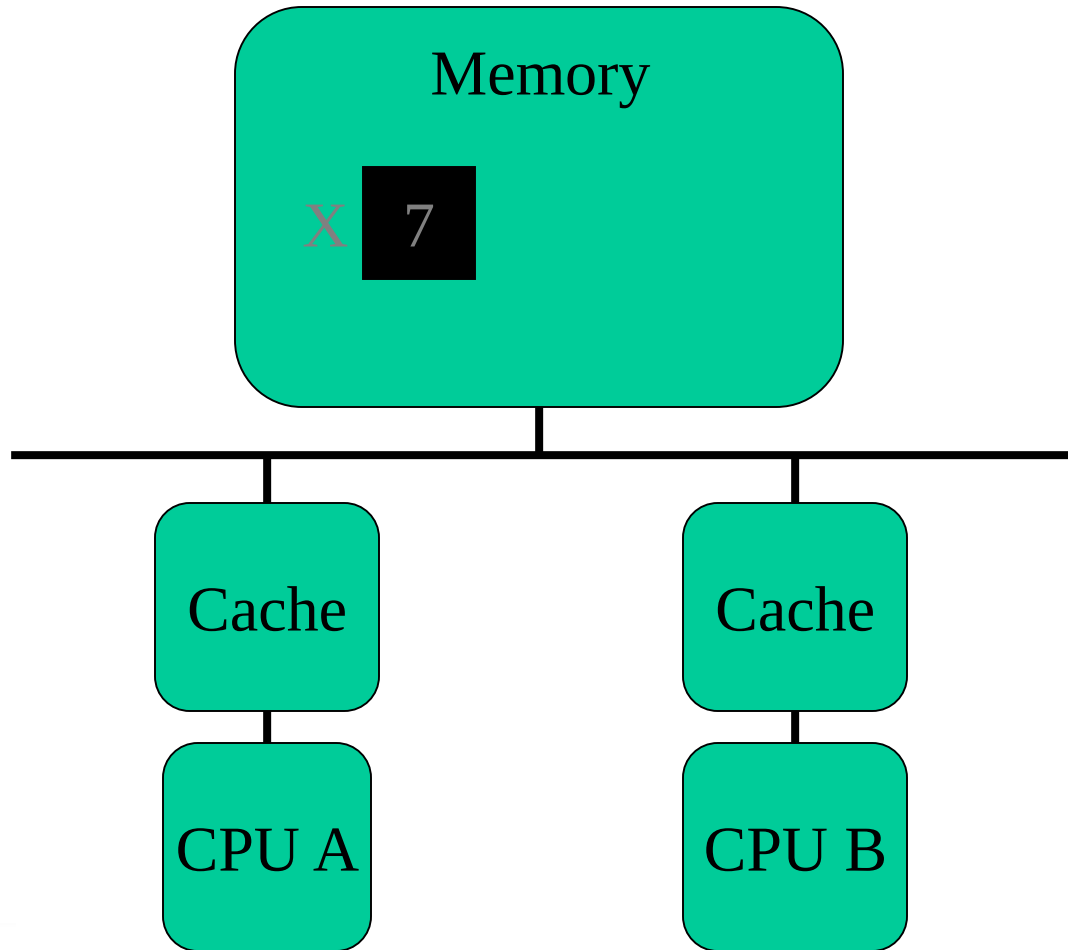
- Cache Coherence problem
- Synchronization problem

Centralized Multiprocessors -- Issues

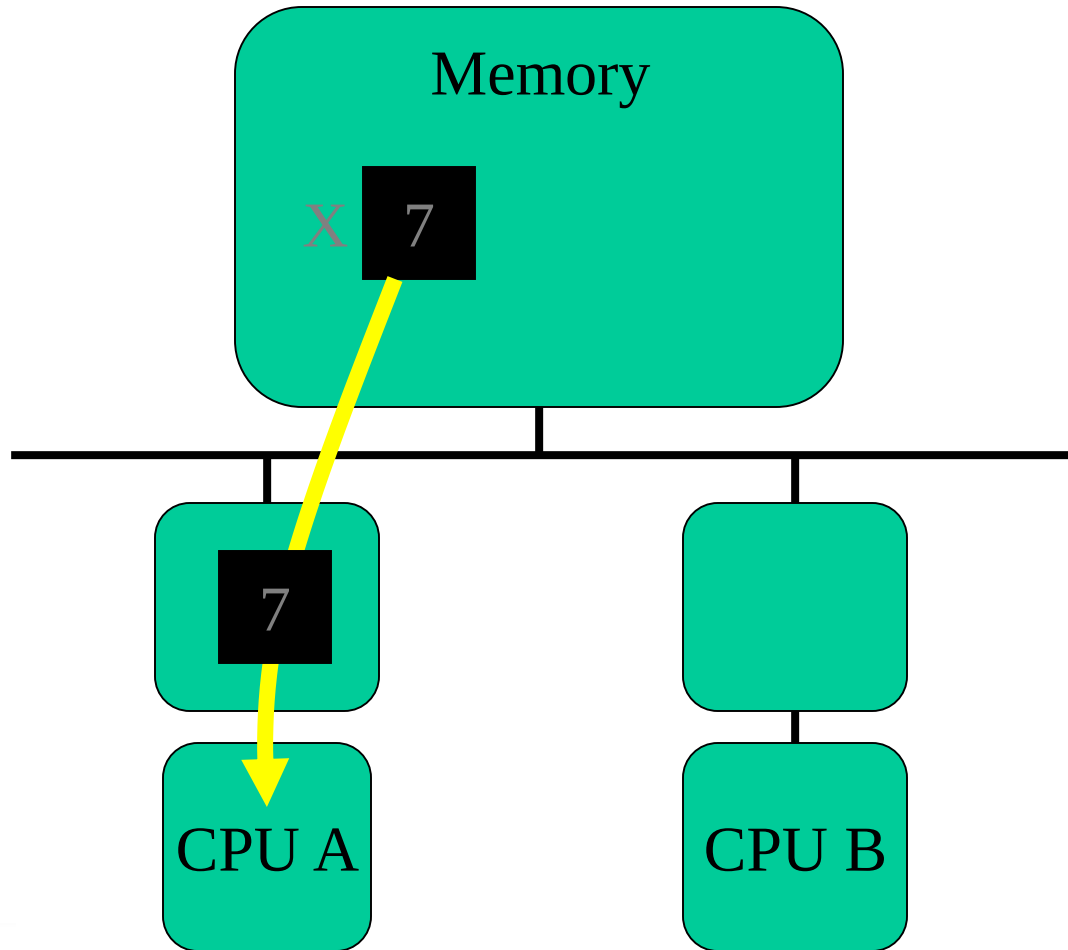
- Cache Coherence problem

- Different processors might have different values for the same memory location (for shared data) – due to caches.
- i.e., two processors caching different values of the same memory location.
- NOTE: Cache coherence problem is not possible if only one CACHE is available.
- Caches are expensive. Hence, individual cores get different caches.

Cache-coherence Problem

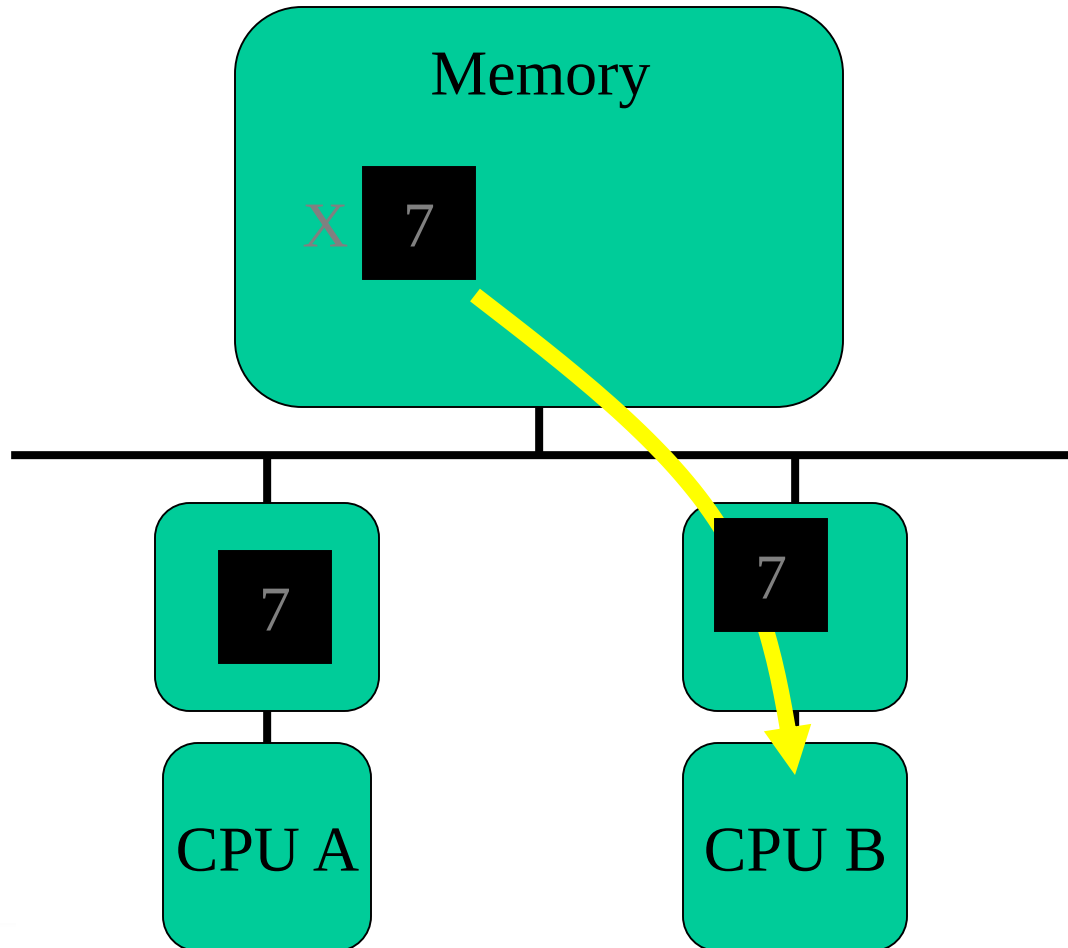


Cache-coherence Problem



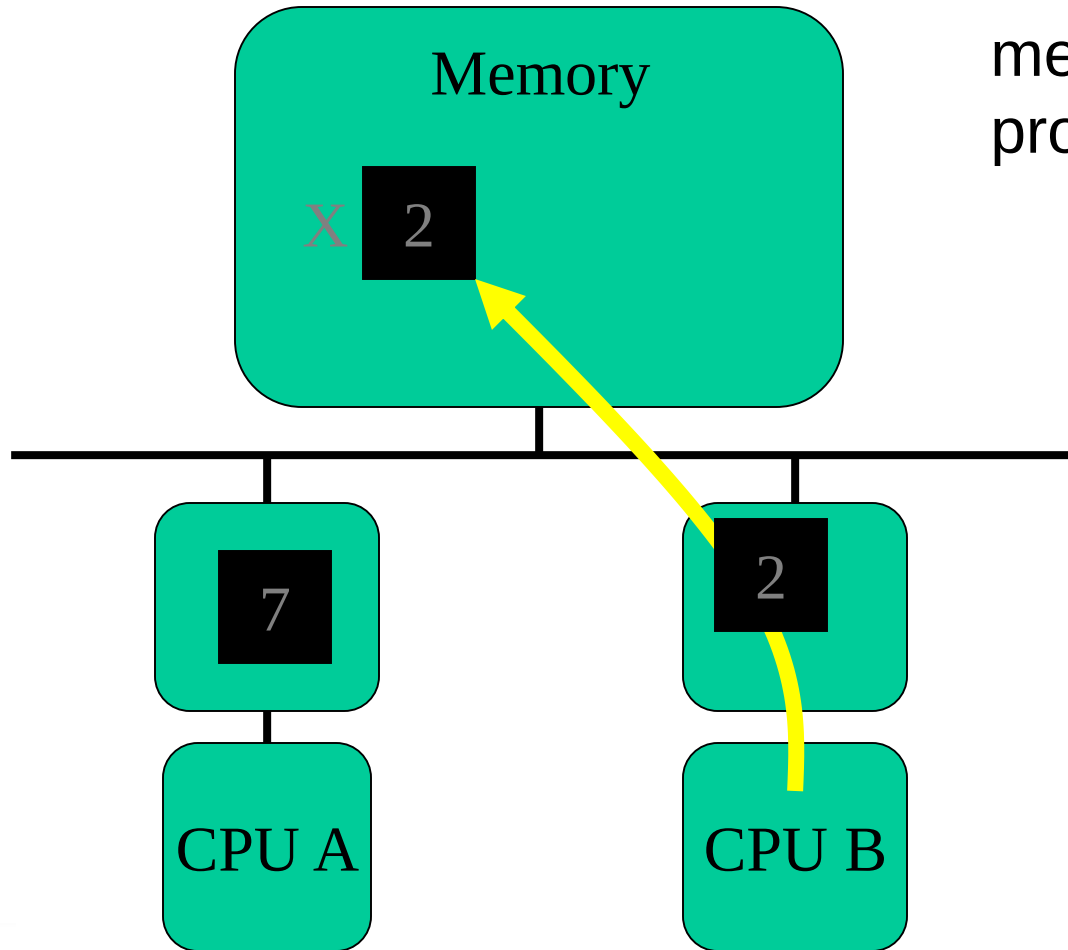
Read from
memory is not a
problem.

Cache-coherence Problem



Cache-coherence Problem

Write to main memory is a problem.



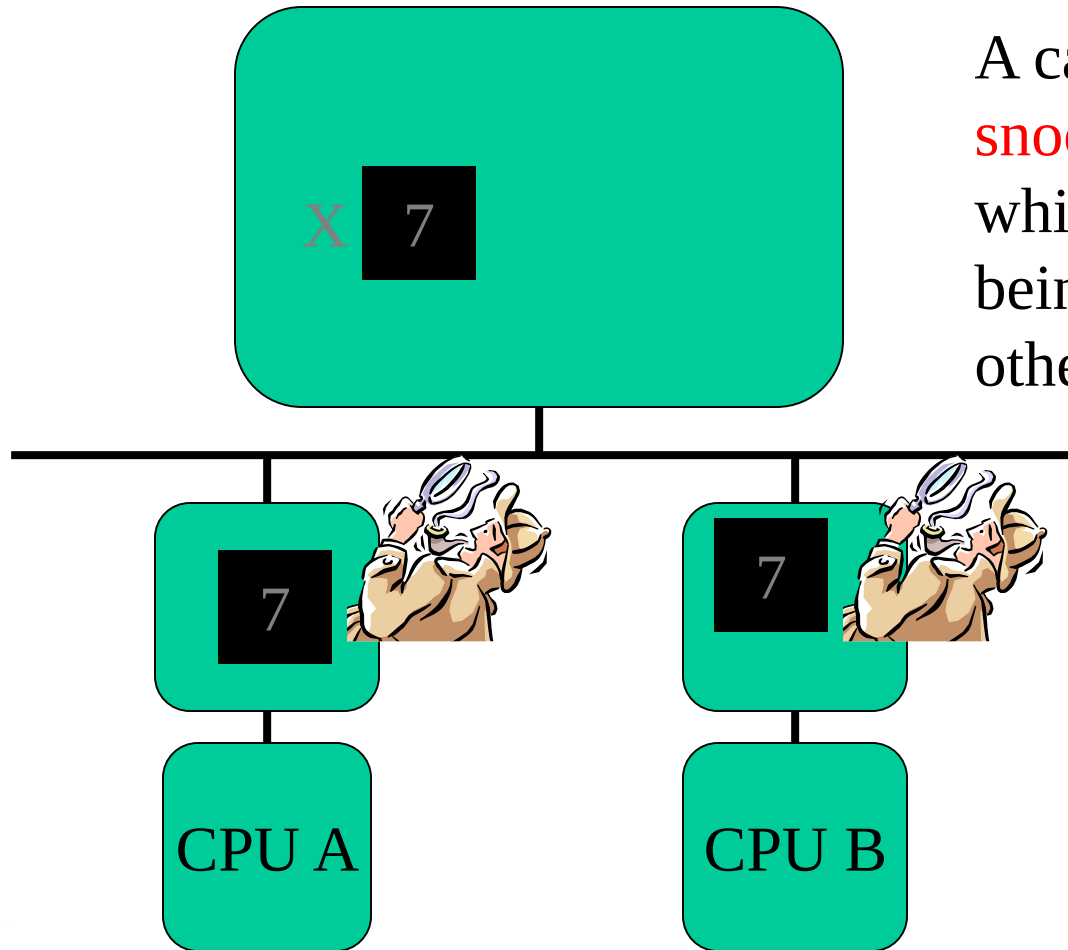
Cache Coherence Problem -- SUMMARY

- How to solve the cache coherence problem?
 - Via. Snoopy protocols.
- Snoopy protocols
 - Here, the cache controller of CPUs monitors (Snoops) the bus to identify which cache blocks are requested by other CPUs.
 - Two approaches are possible under Snoopy Protocol
 - Write Invalidate
 - Write Update
 - Write Invalidate

NOTE: Before the write occurs, all copies of the data items cached by other processors are **invalidated (WRITE INVALIDATE)**.
 - Write Update

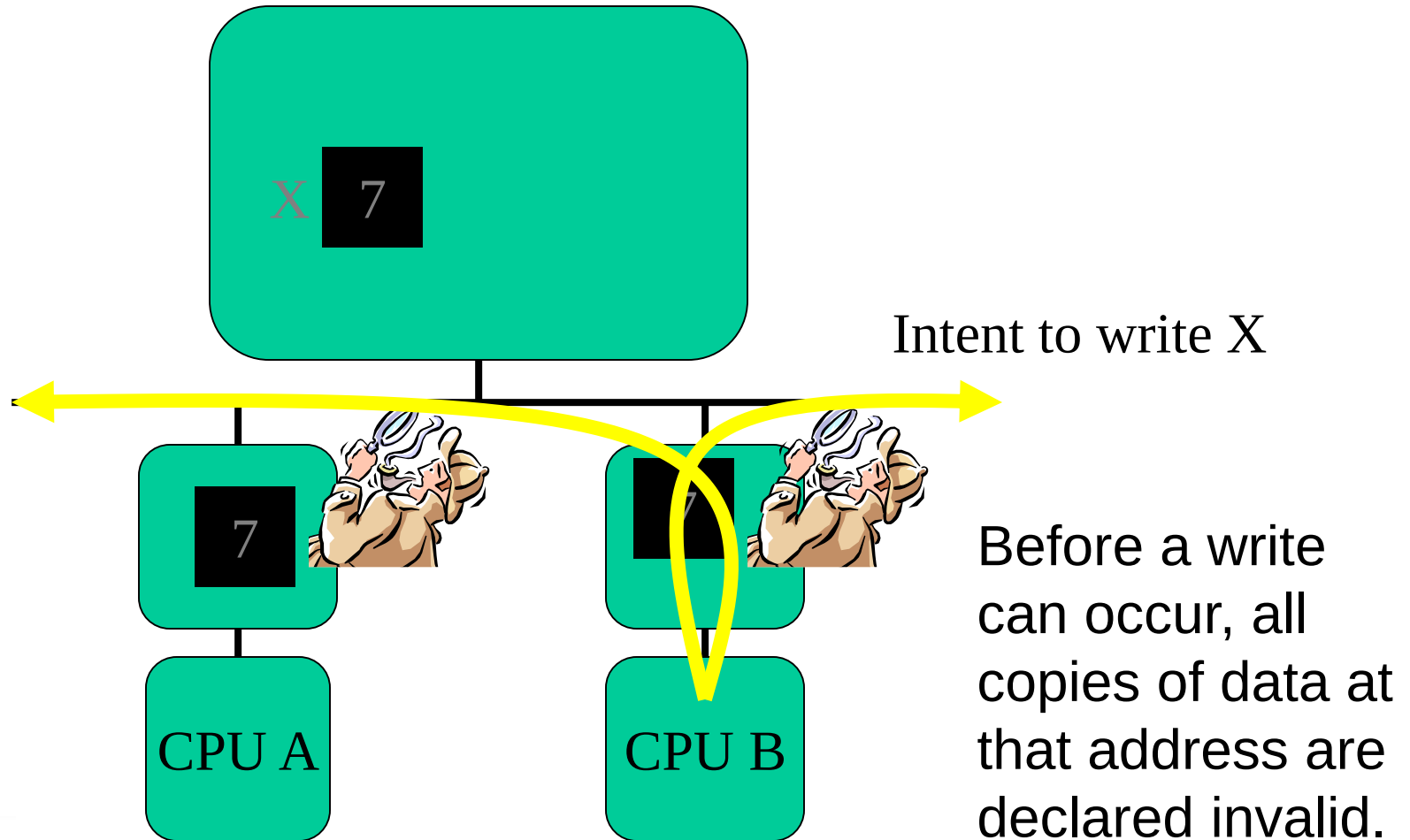
NOTE: Before the write occurs, all copies of the data items cached by other processors are updated

Write Invalidate Protocol

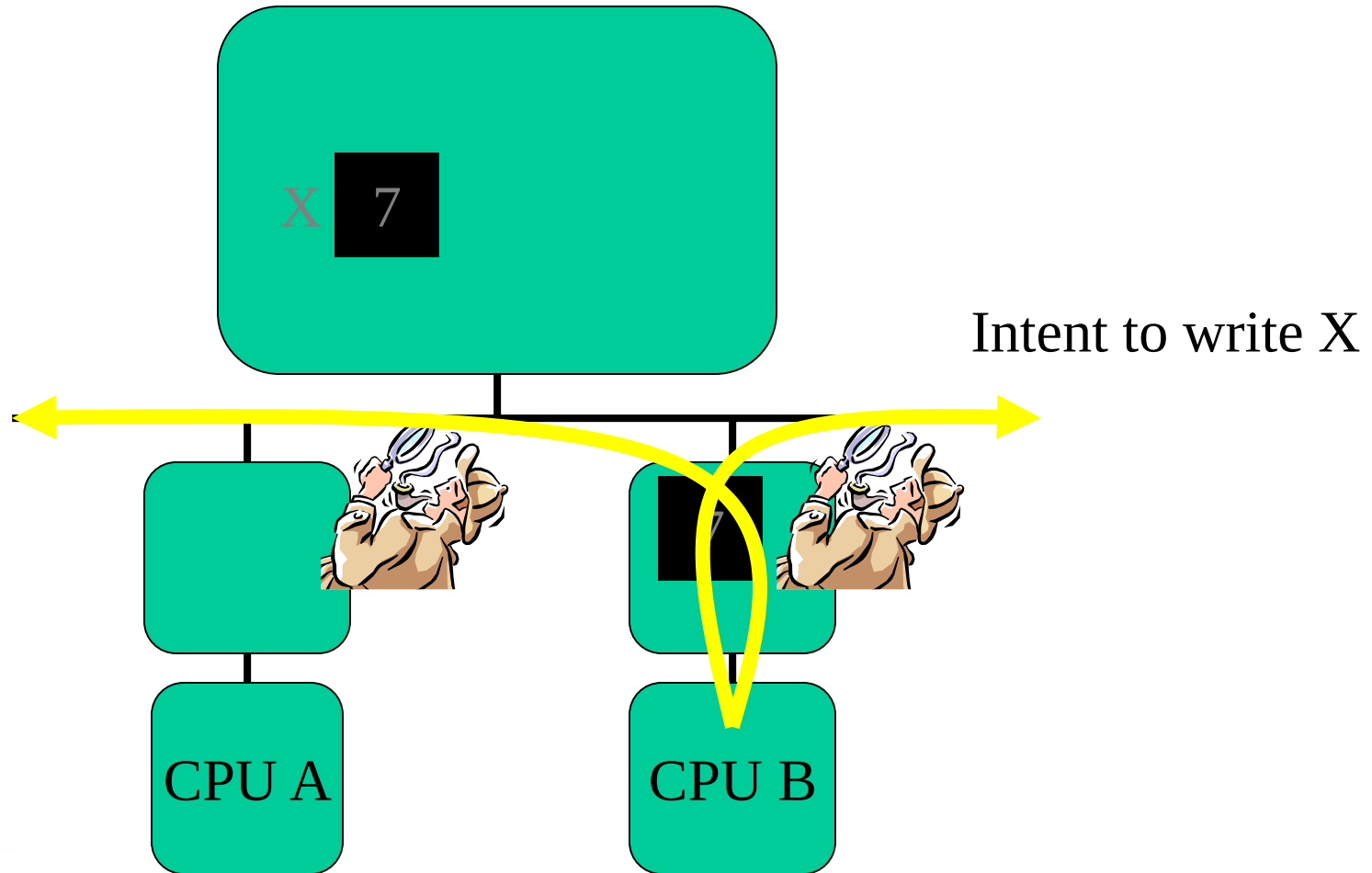


A cache control monitor **snoops** the bus to see which cache block is being requested by other processors.

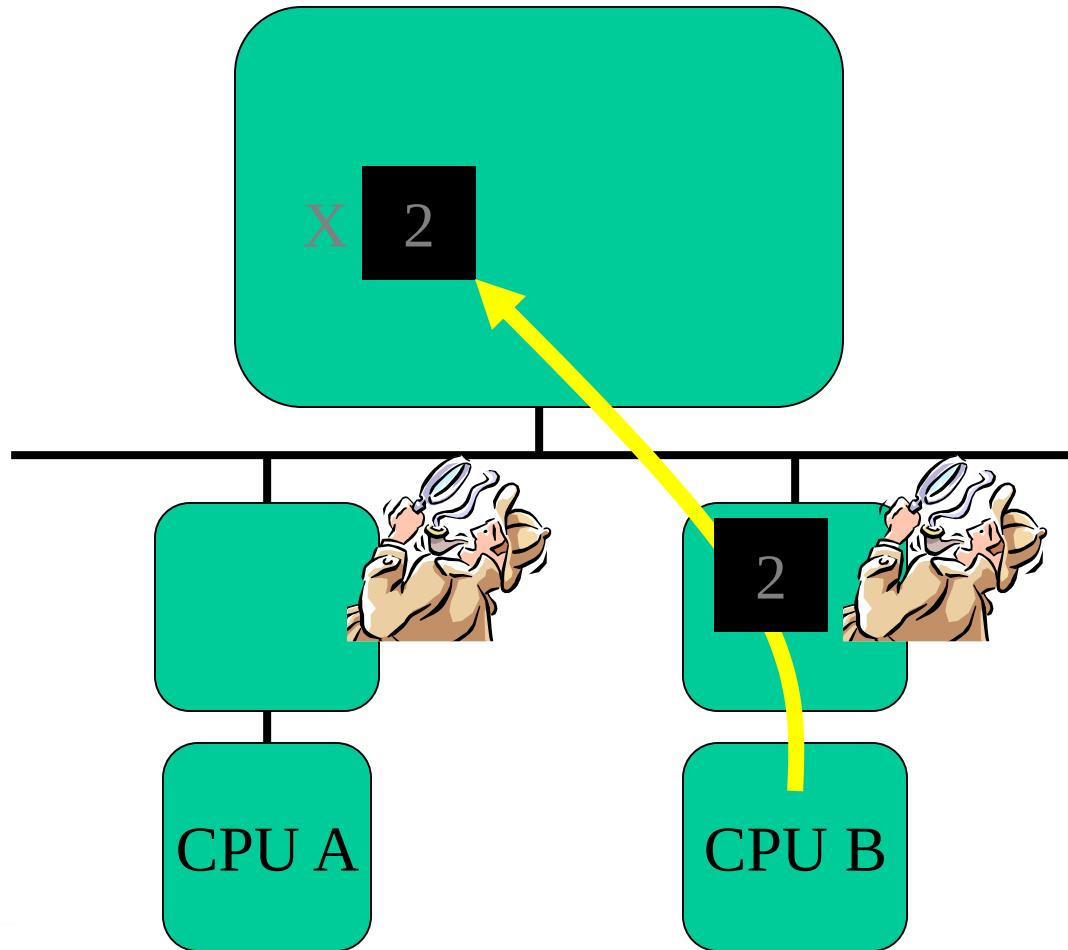
Write Invalidate Protocol



Write Invalidate Protocol



Write Invalidate Protocol



When another processor tries to read from this location in cache, it receives a **cache miss** error and will have to refresh from main memory.

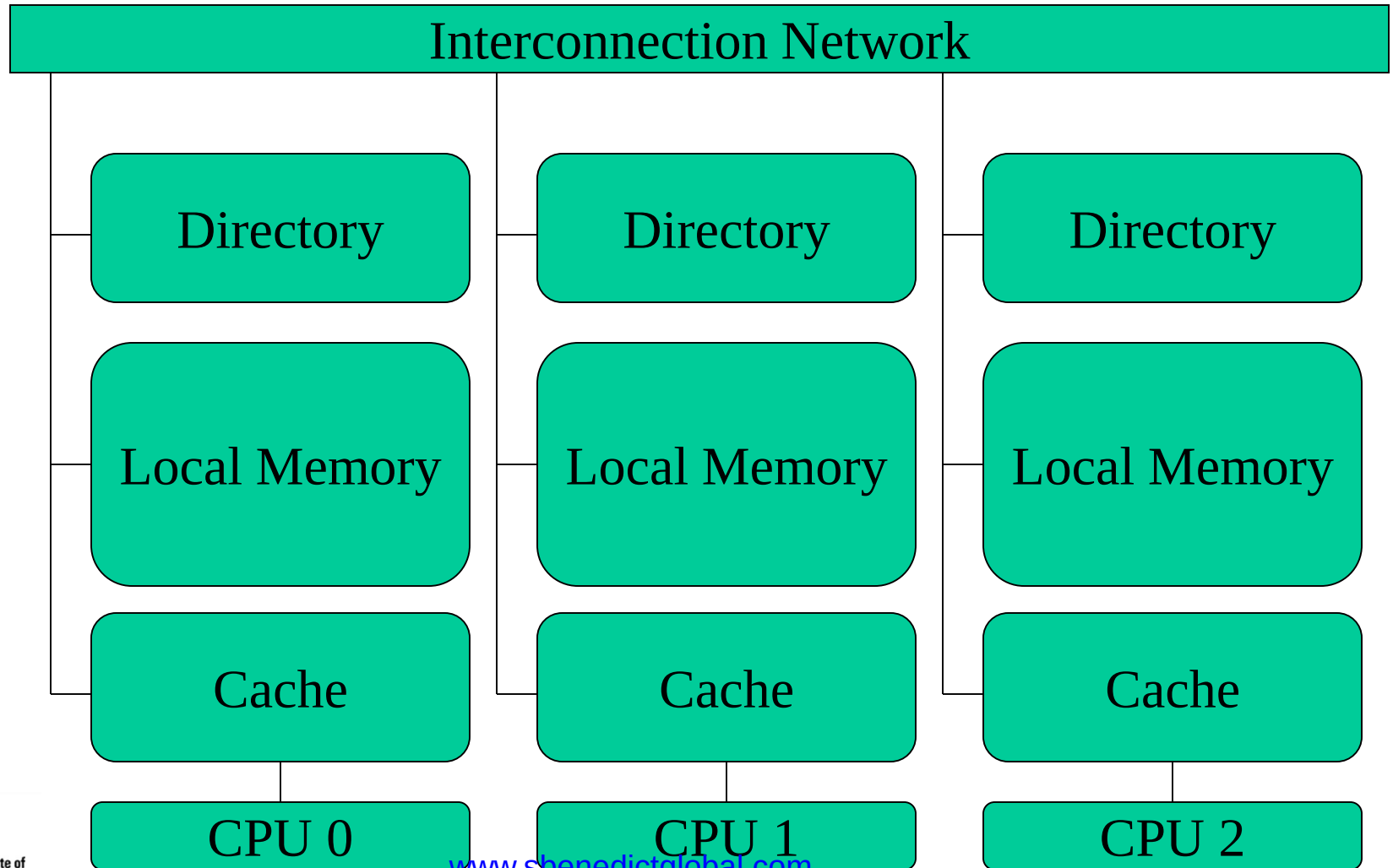
Synchronization Problem

- REMEMBER: Two problems arise while dealing with the shared data in the centralized multiprocessors
 - Cache Coherence problem
 - Synchronization problem
- To solve synchronization problem, we could choose either the following:
 - Mutual Exclusion – i.e., atmost one processor can be engaged in an activity at a time.
 - Barrier Synchronization – i.e, a barrier point is maintained so that all processes wait at this point.

Cache Coherence @ Distributed Multiprocessors

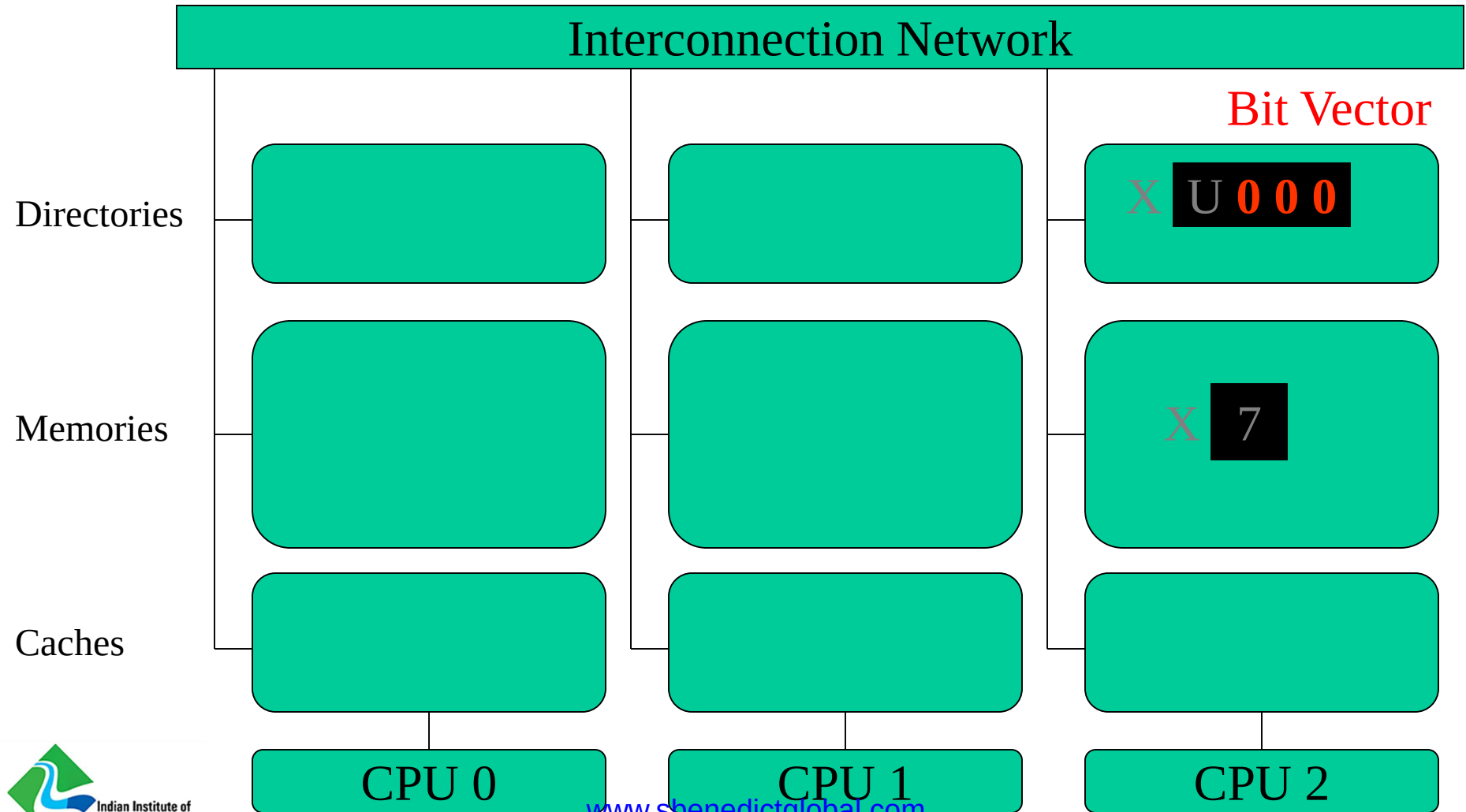
- Snoopy protocols would not scale well for distributed multiprocessor architectures.
- Hence, directory-based protocol is applied.
- Directory-based protocol
 - A single directory, which contains the **shared information** of every cache blocks, is available.
 - What is contained in the shared information of the directory?
 - A few sharing status information is available, such as,
 - Whether uncached? i.e., not available in any processor's cache.
 - Whether shared? – i.e., cached by one or more processors (and, the copy in the memory is correct.)
 - Whether exclusive? – i.e., cached by exactly one processor that has written the block (so that the copy in the memory is obsolete).

Directory-based Protocol

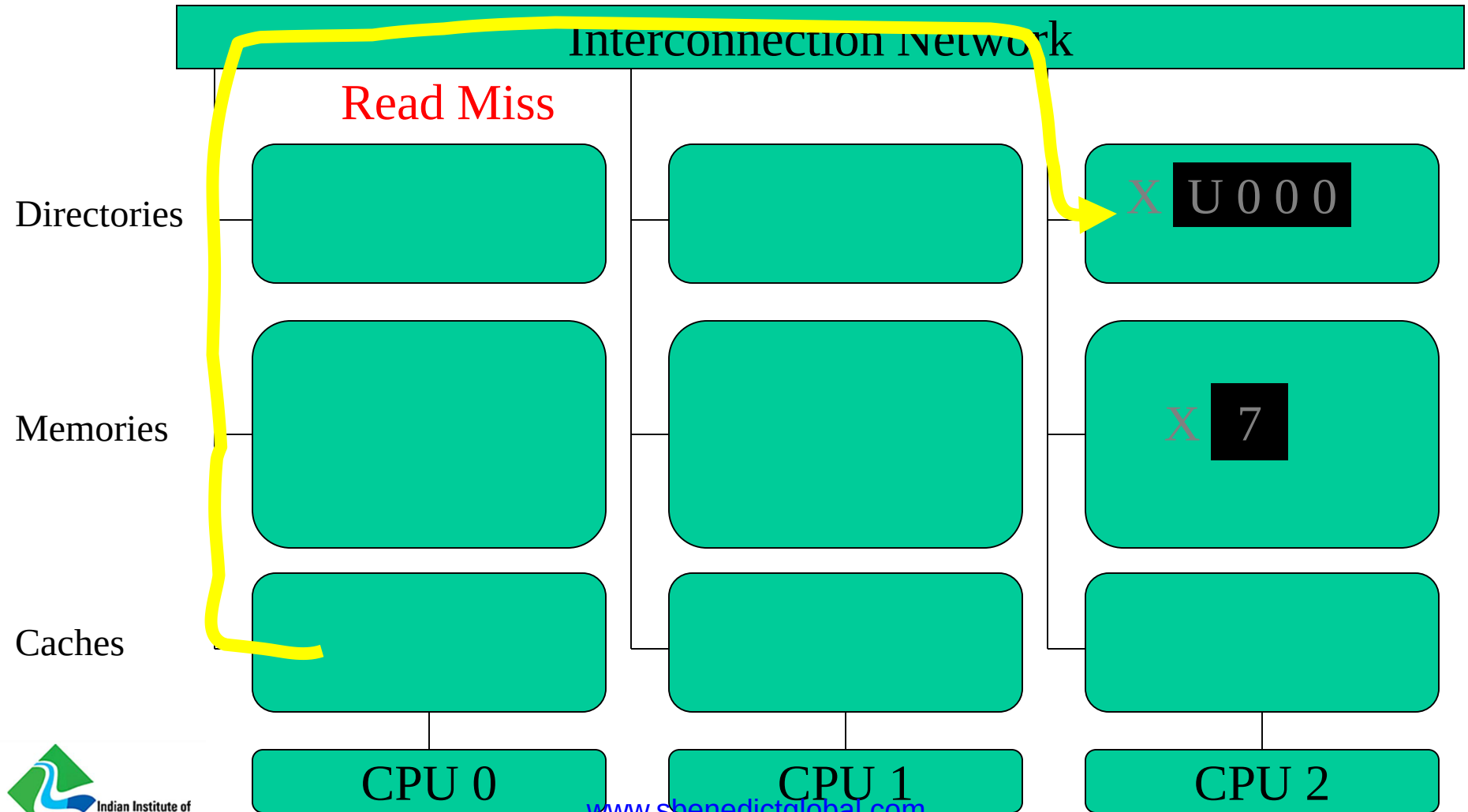


Directory-based Protocol

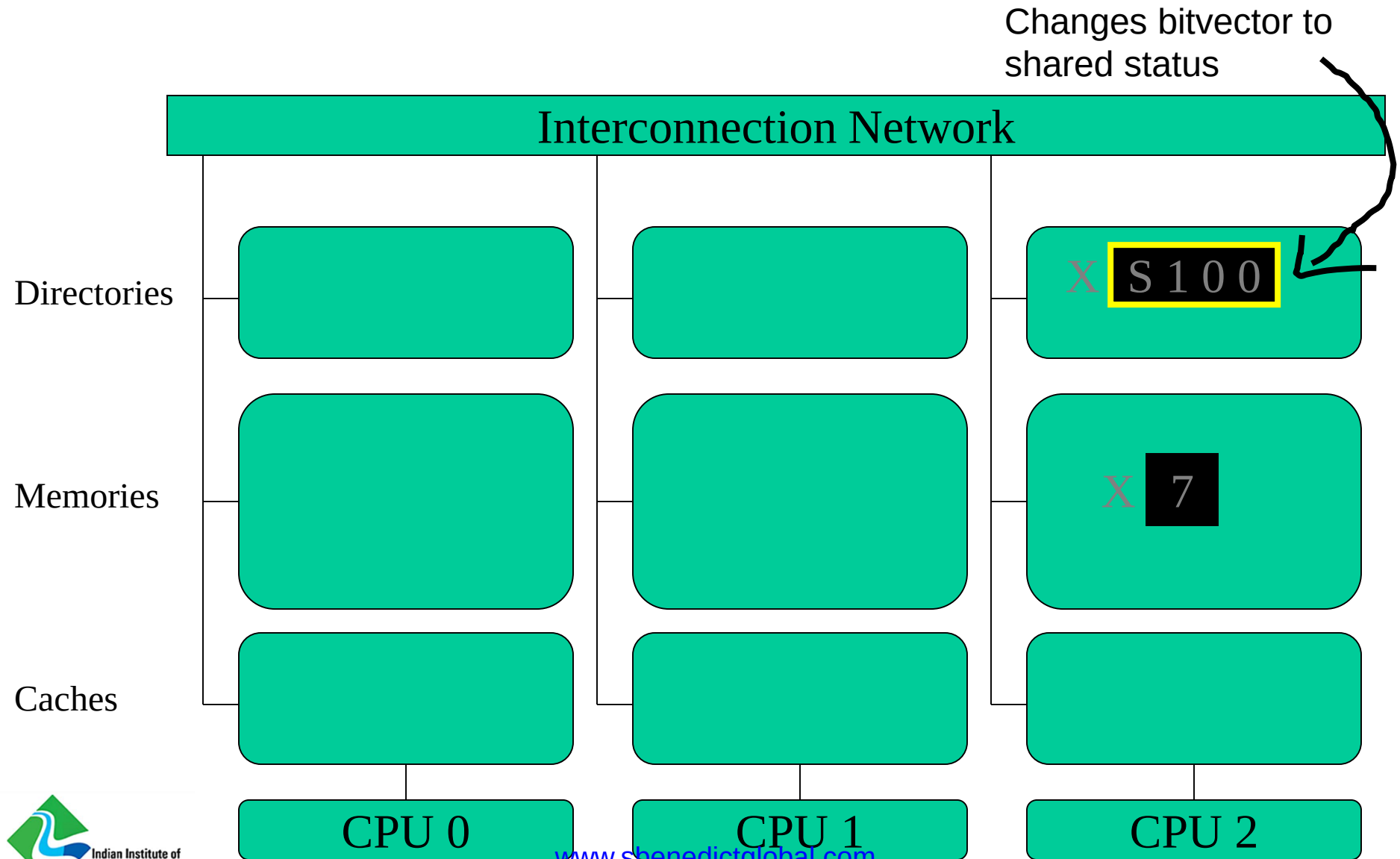
Case 1: CPU0 reads data from the memory of CPU 2



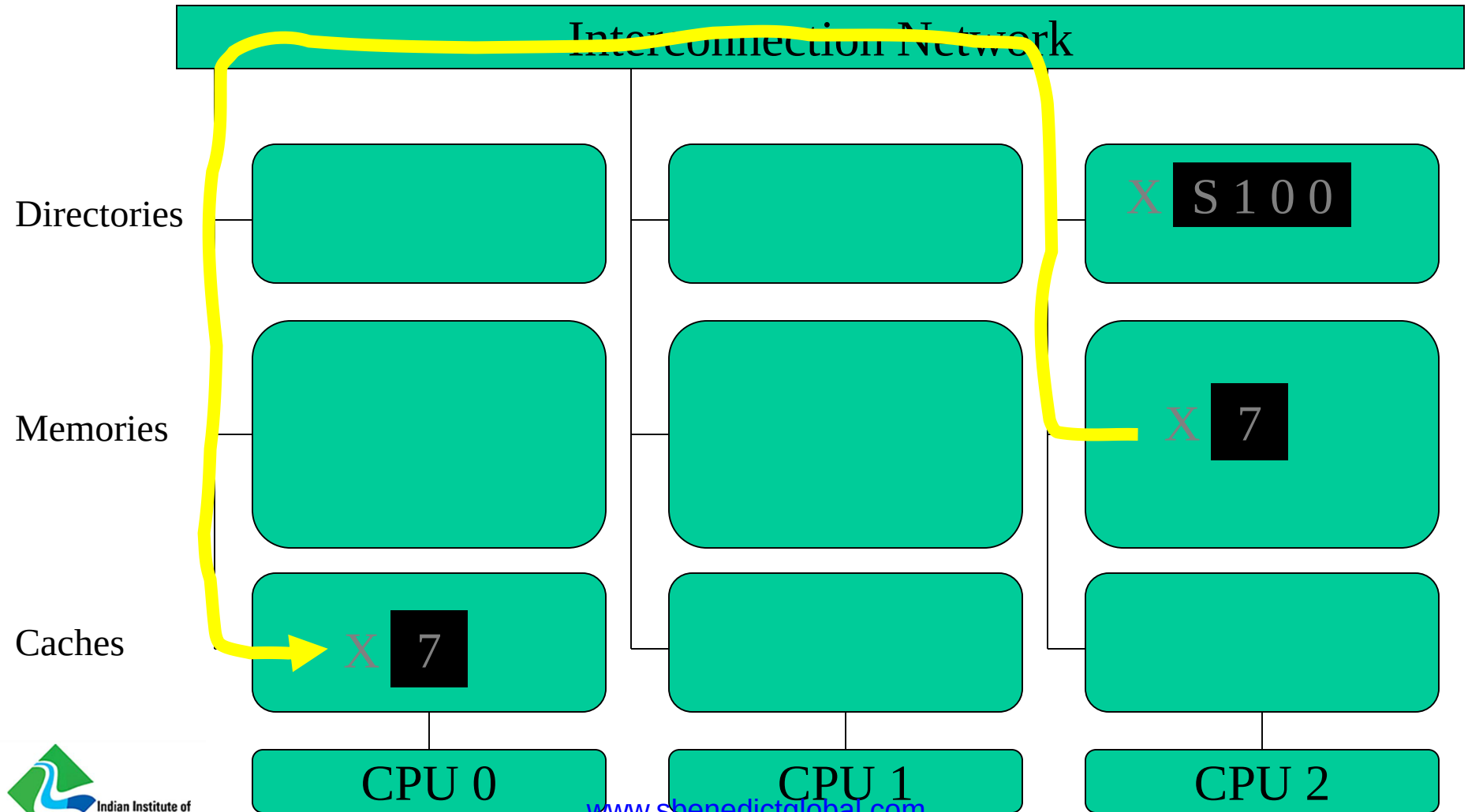
Directory-based Protocol



Directory-based Protocol

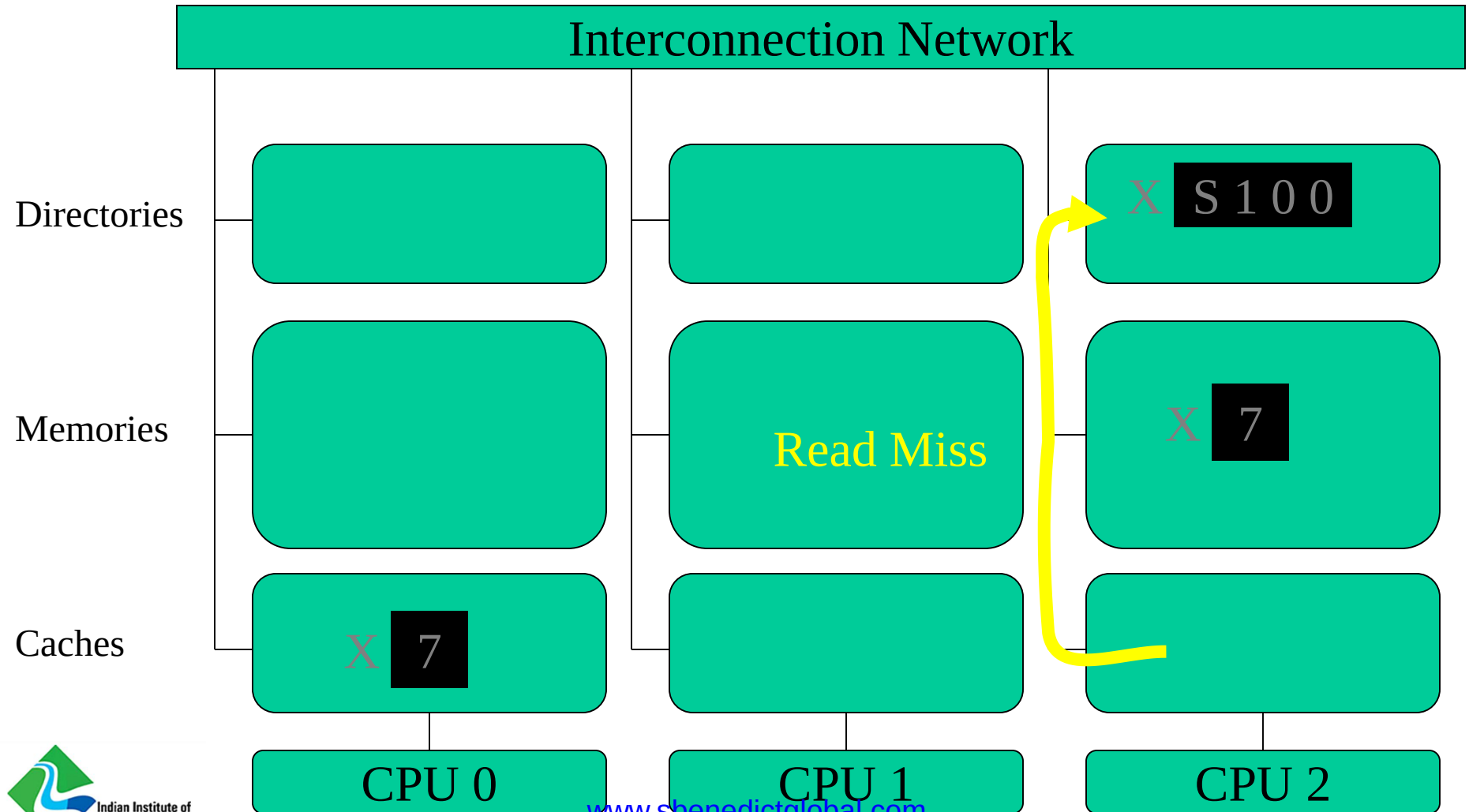


Directory-based Protocol

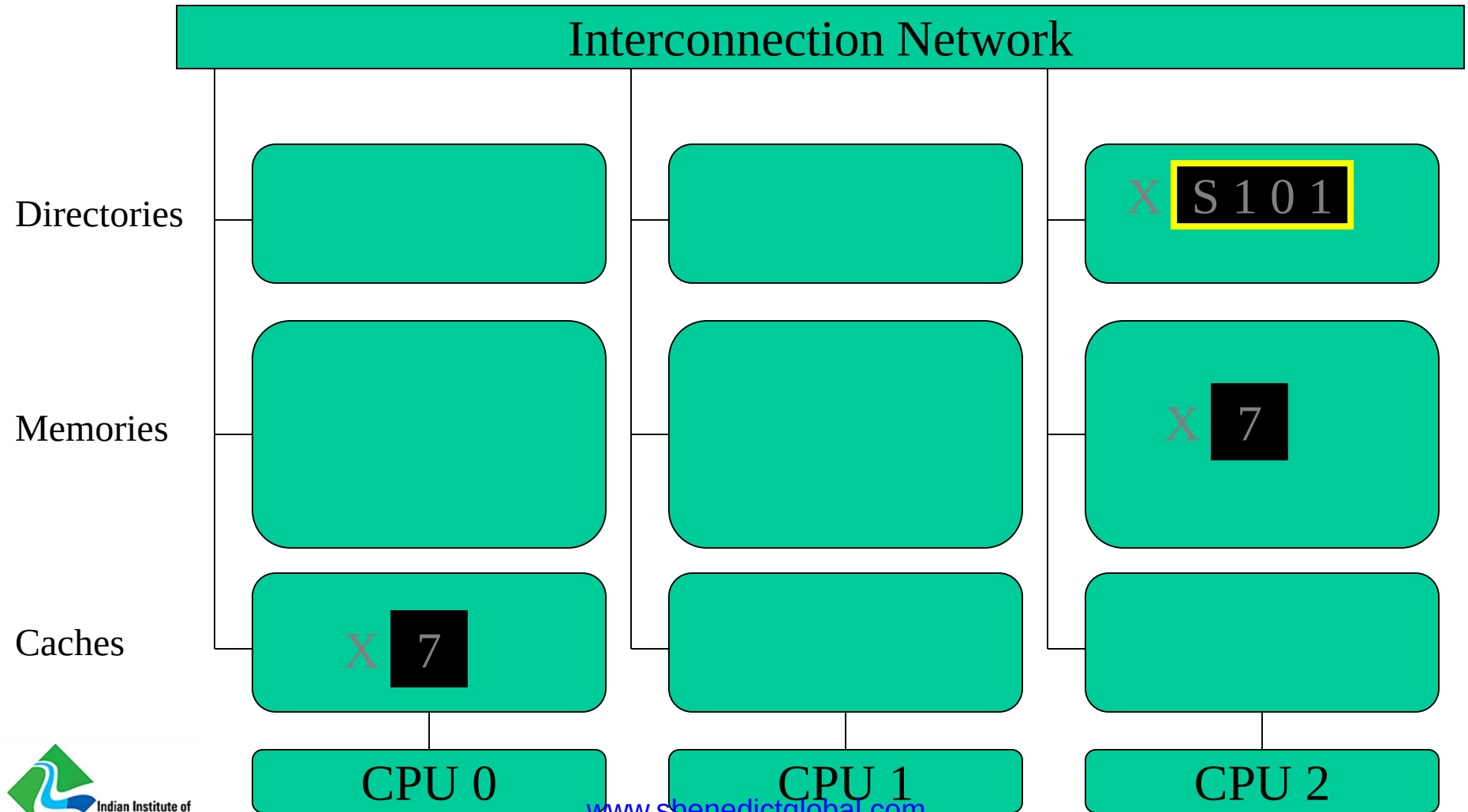


Directory-based Protocol

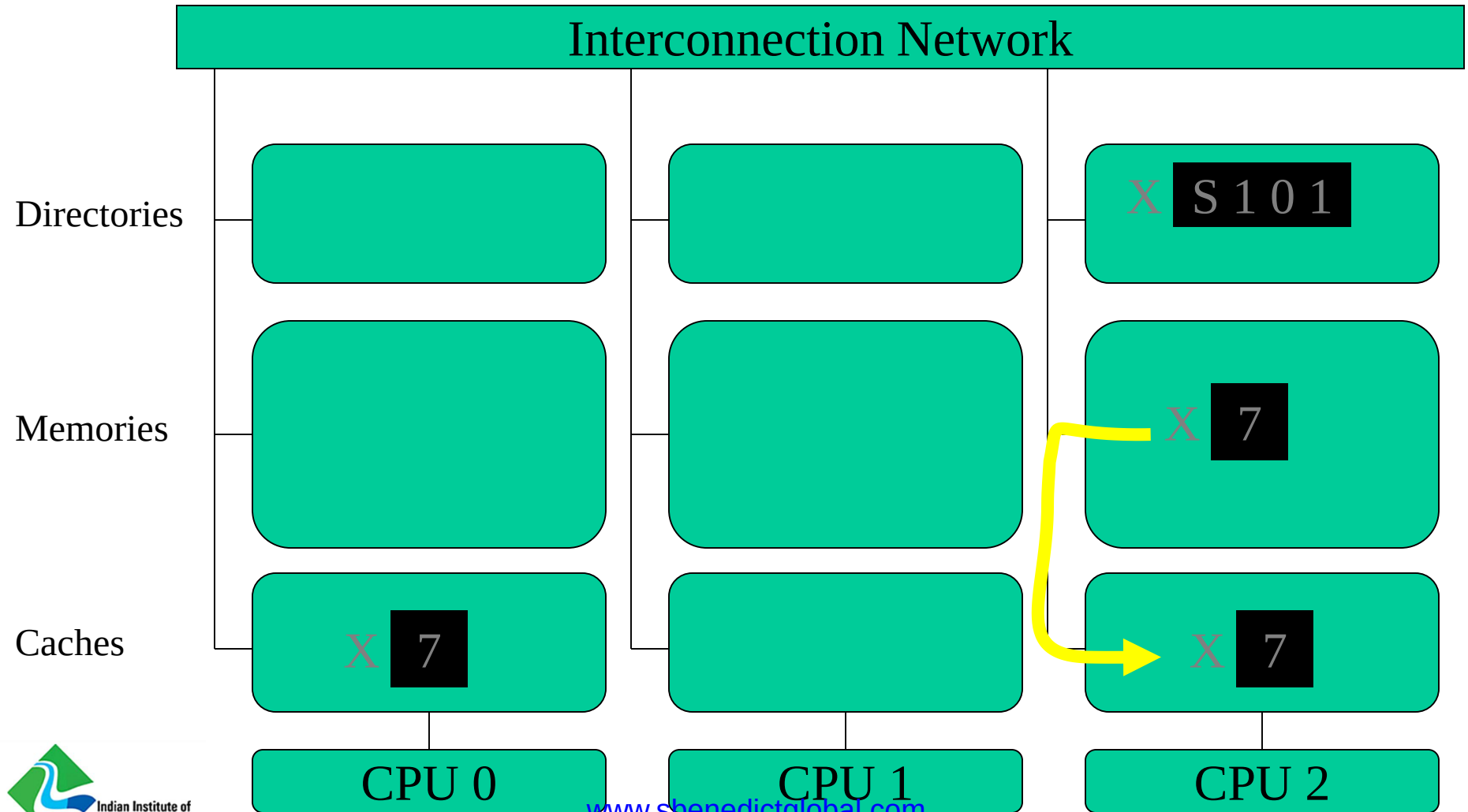
Case 2: CPU2 reads data from the memory of CPU 2



Directory-based Protocol

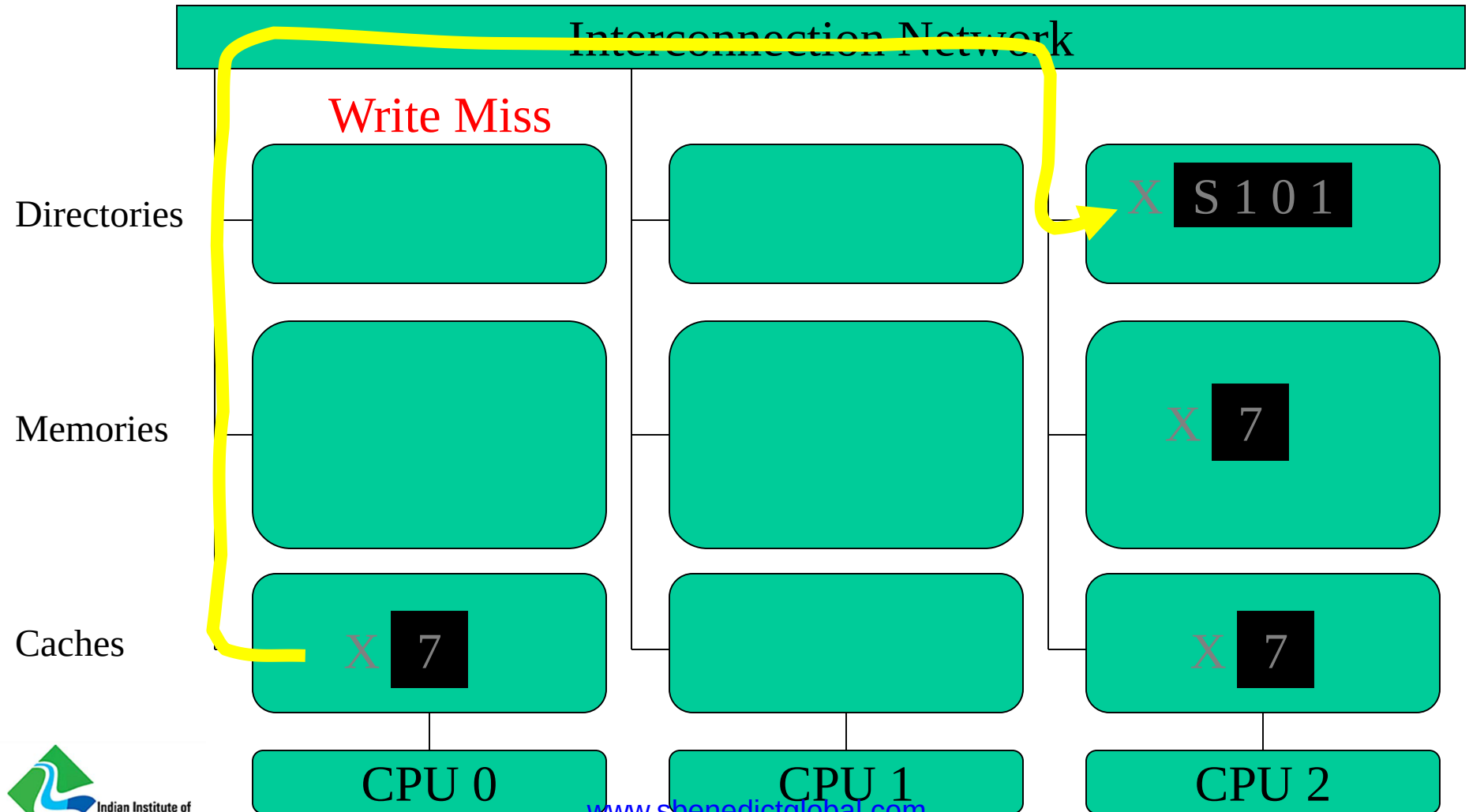


Directory-based Protocol

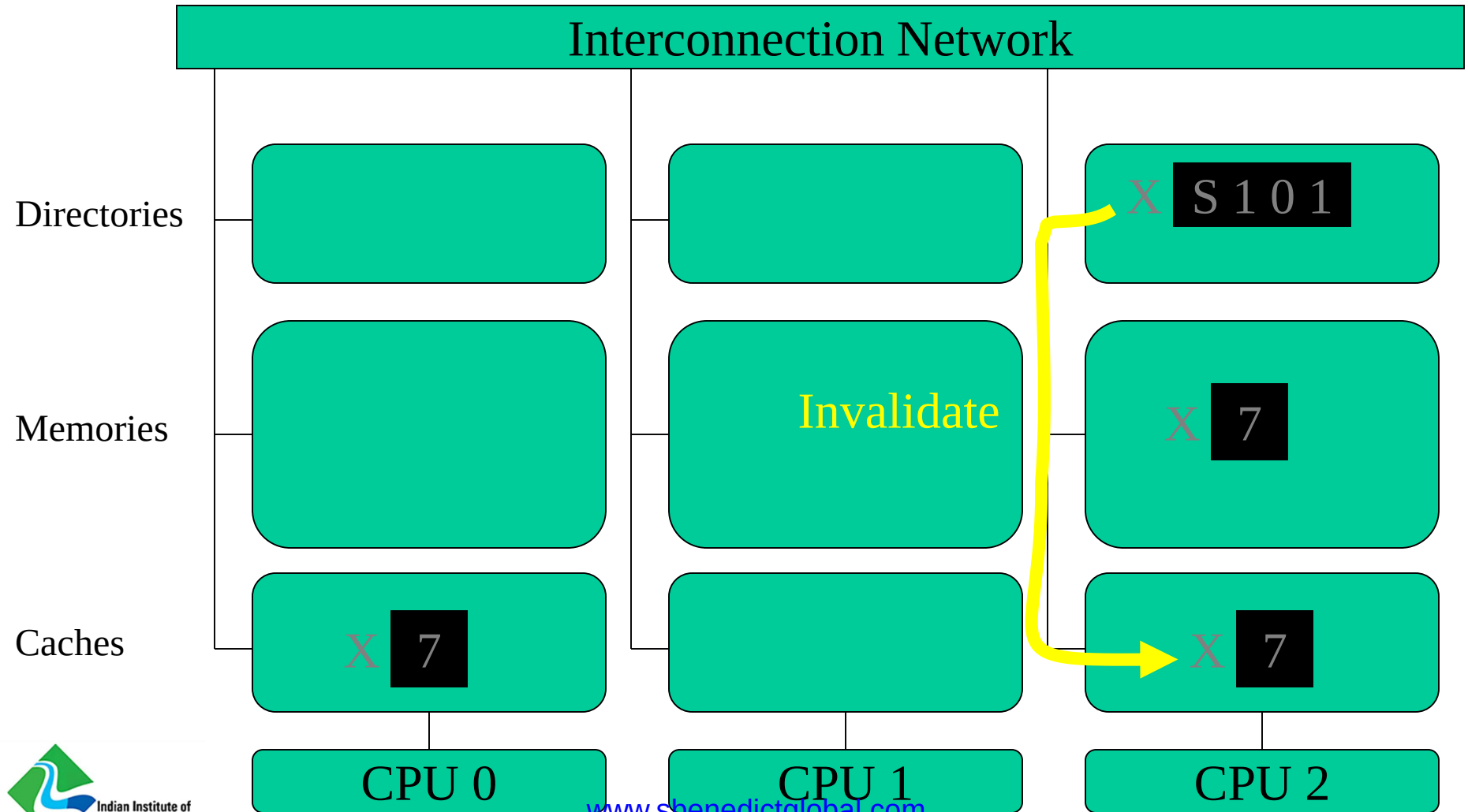


Directory-based Protocol

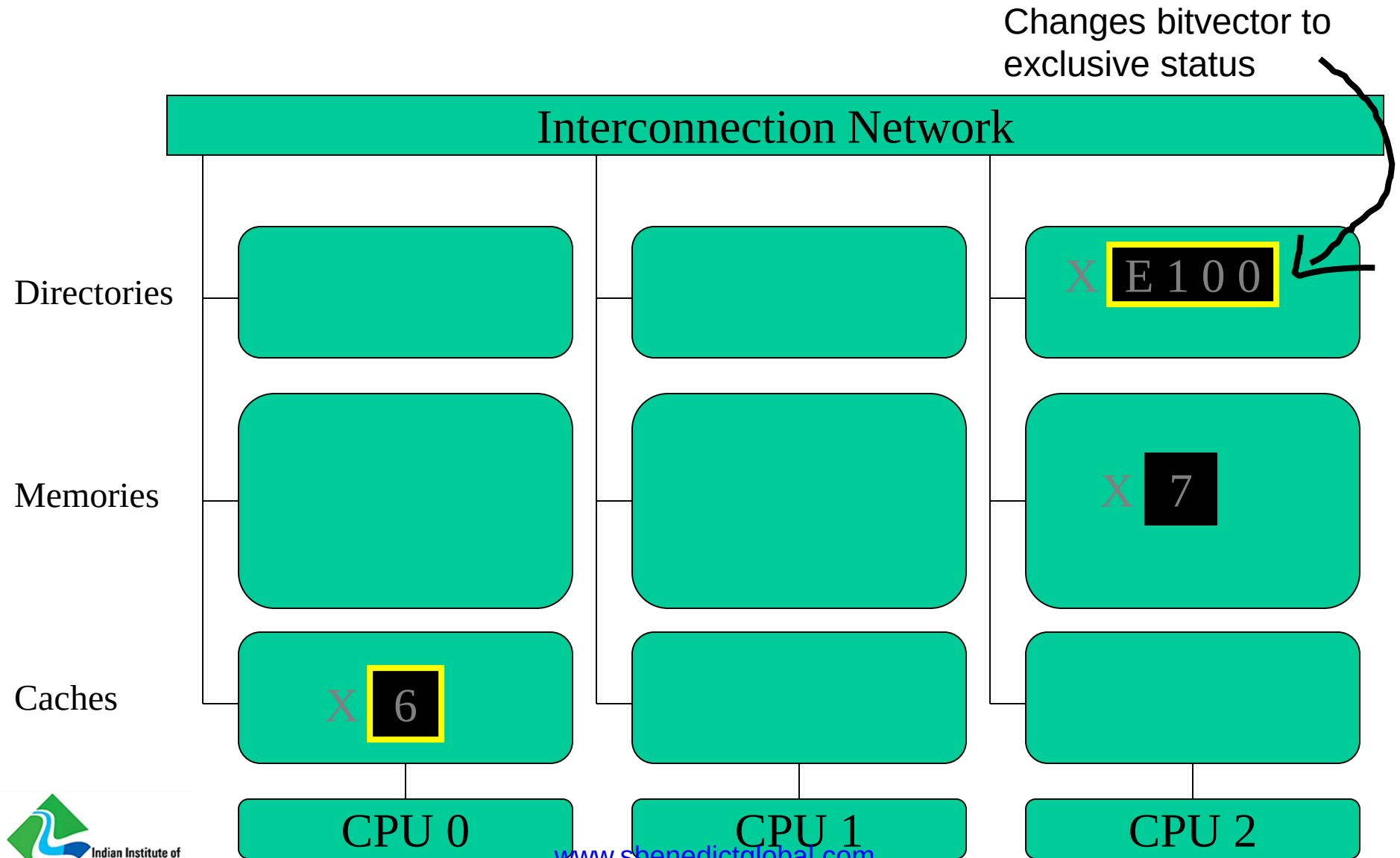
Case 3: CPU 0 writes value 6 to the memory of CPU 2



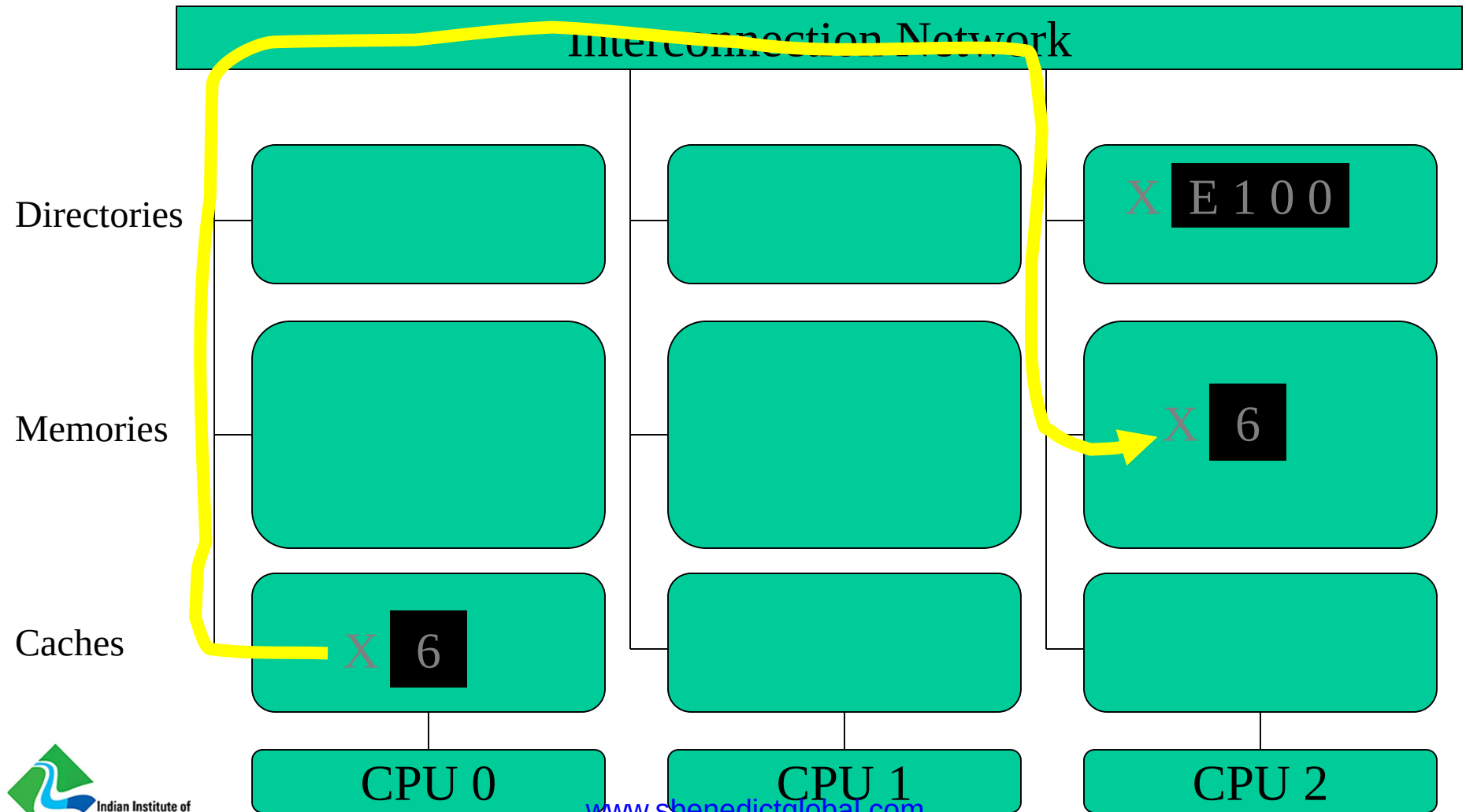
Directory-based Protocol



Directory-based Protocol

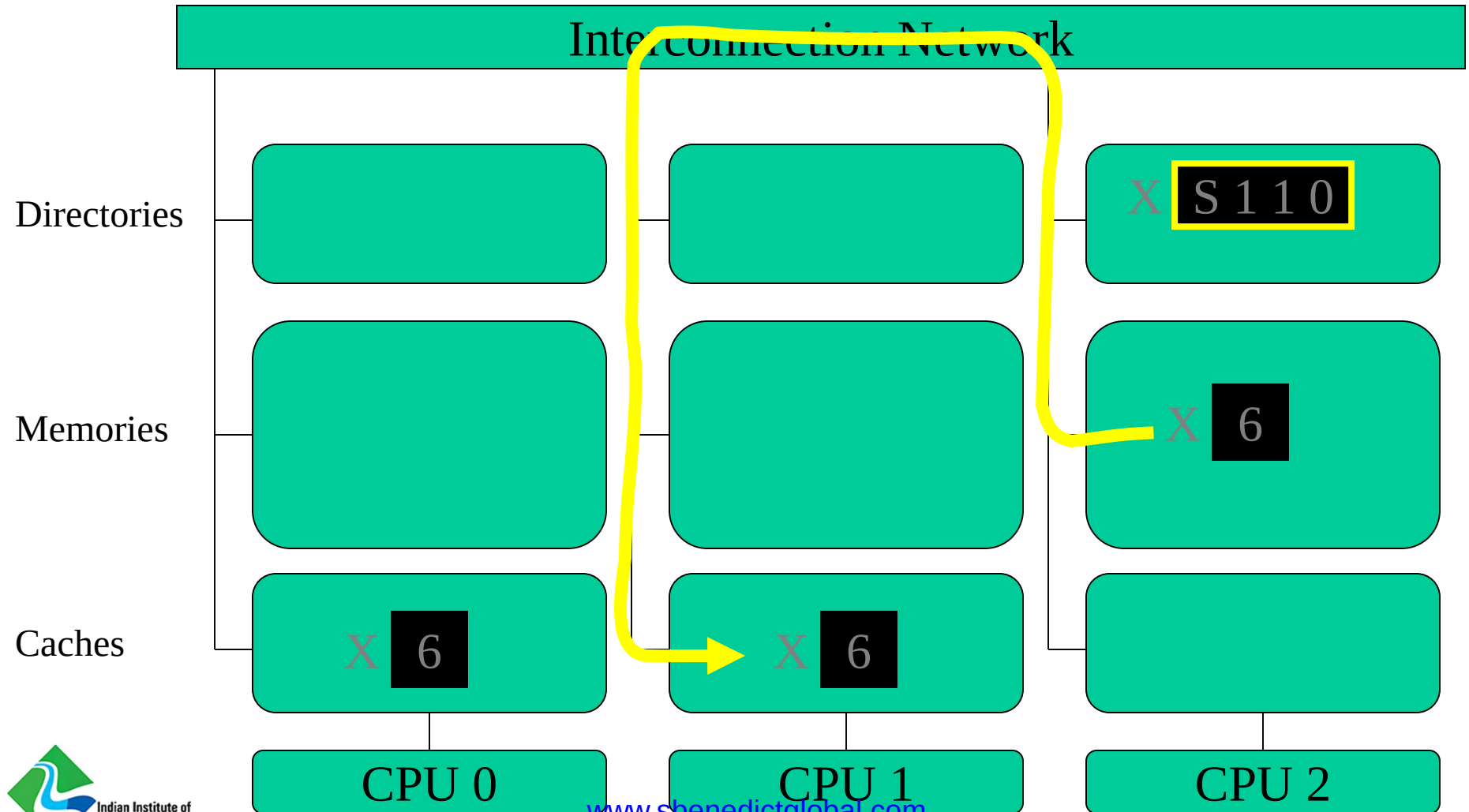


Directory-based Protocol



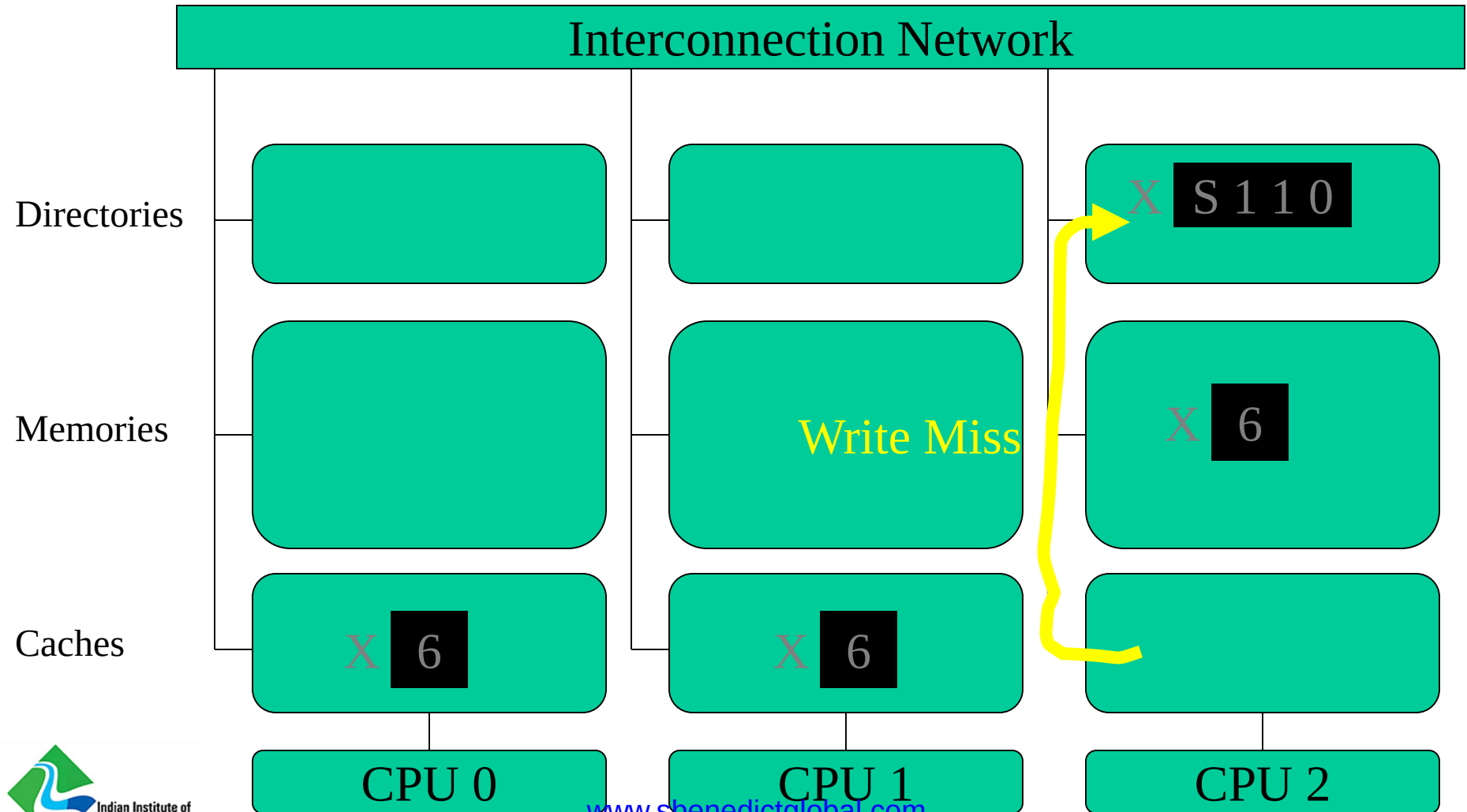
Directory-based Protocol

Case 4: CPU 1 reads data from the memory of CPU 2

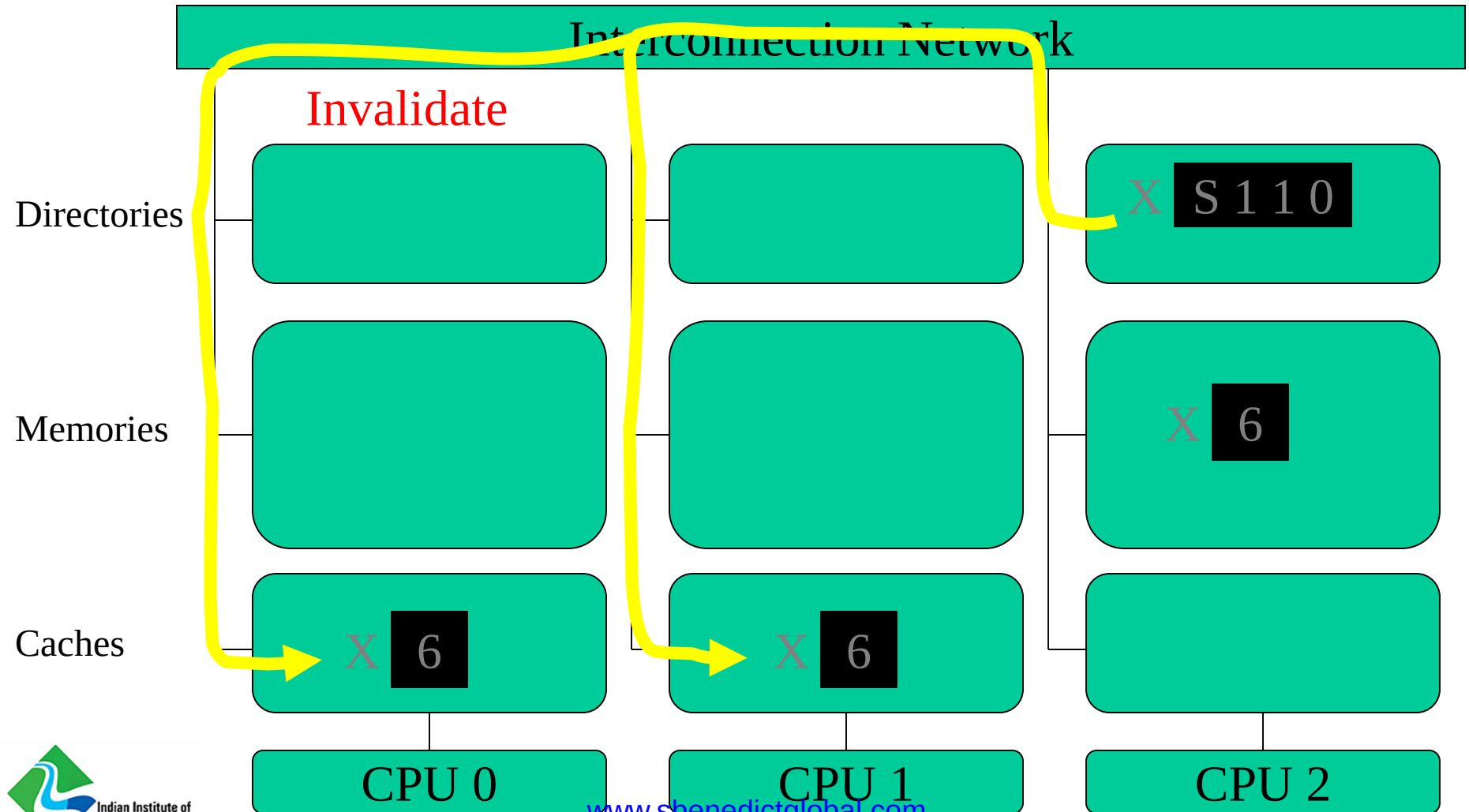


Directory-based Protocol

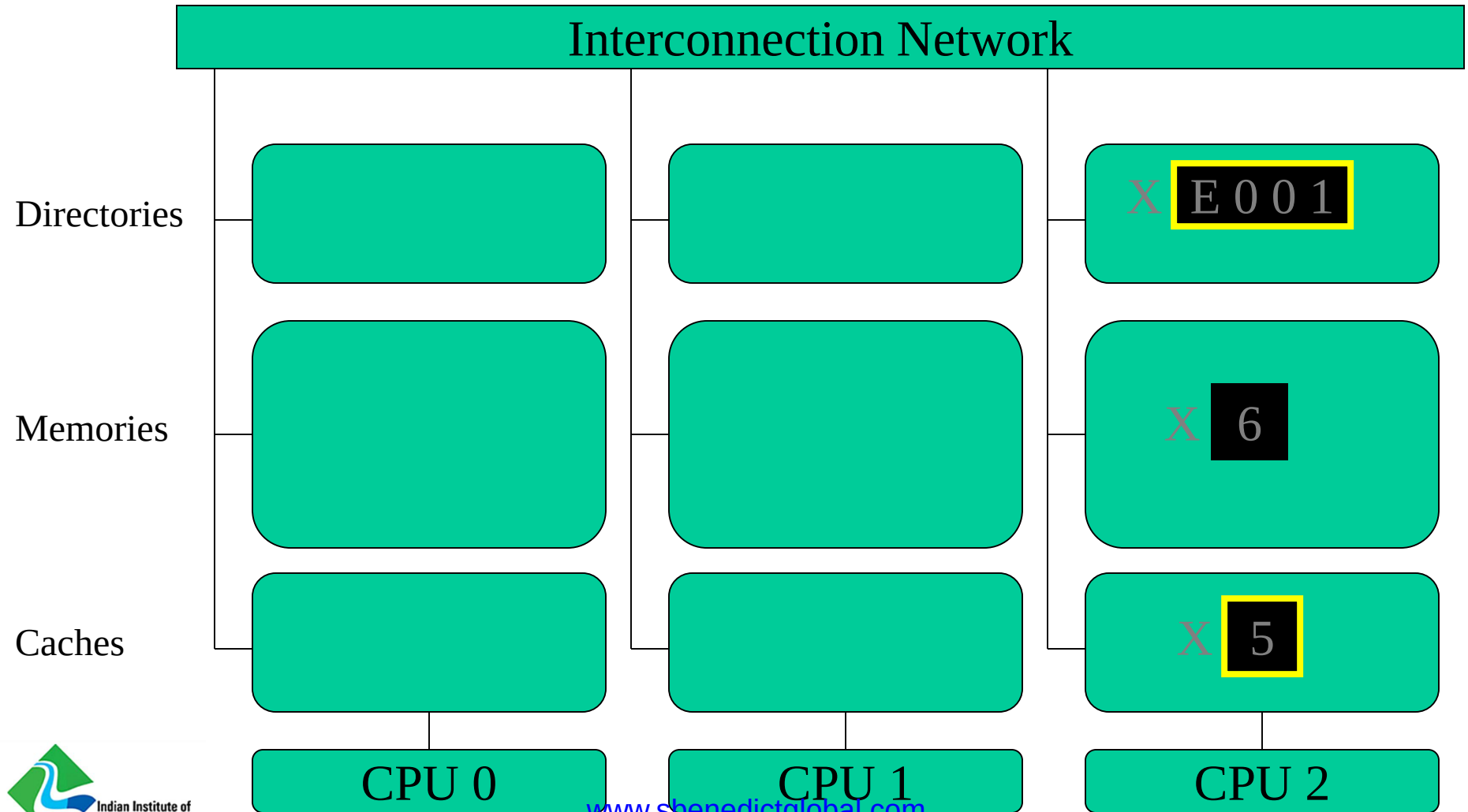
Case 5: CPU2 writes data 5 to memory of CPU 2



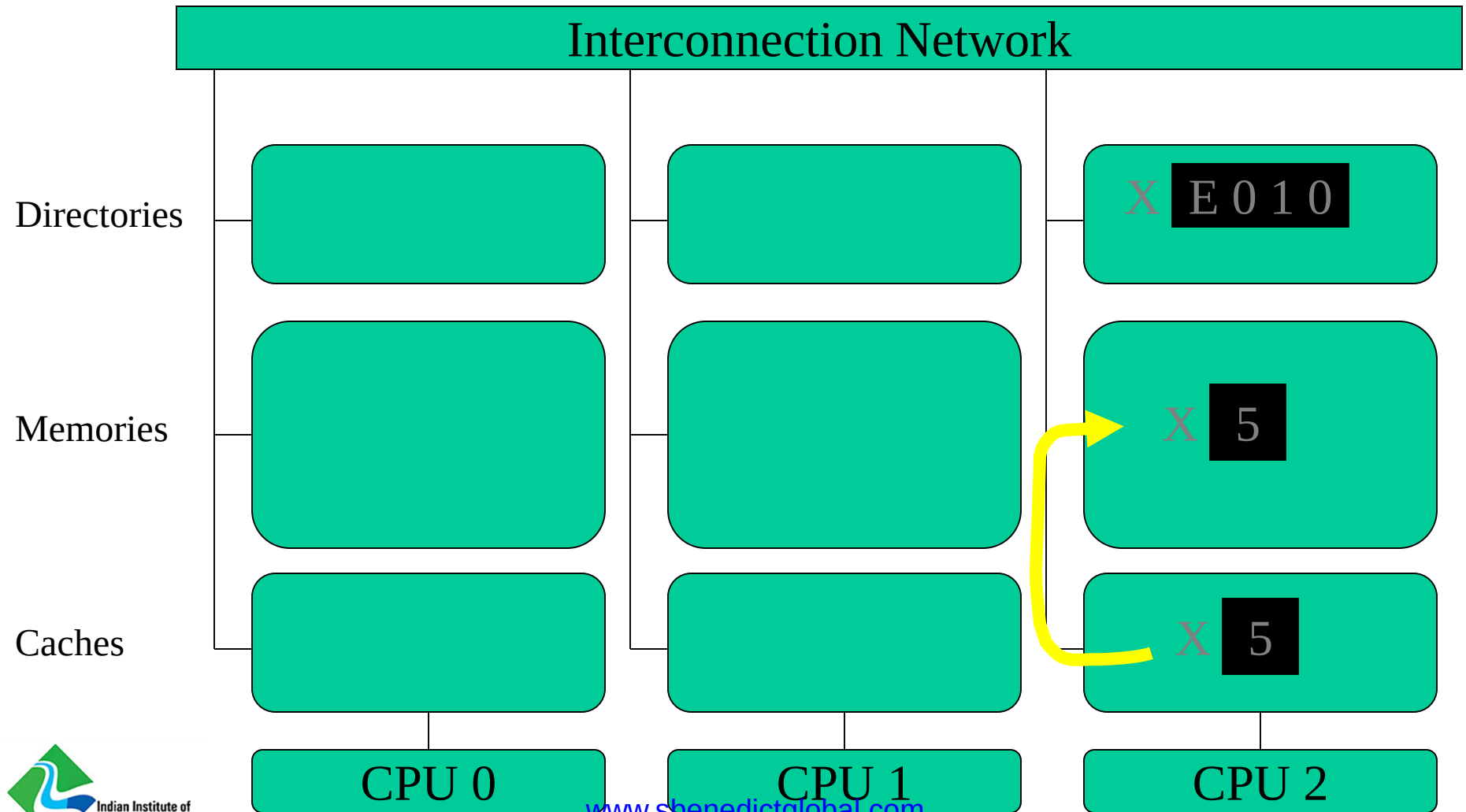
Directory-based Protocol



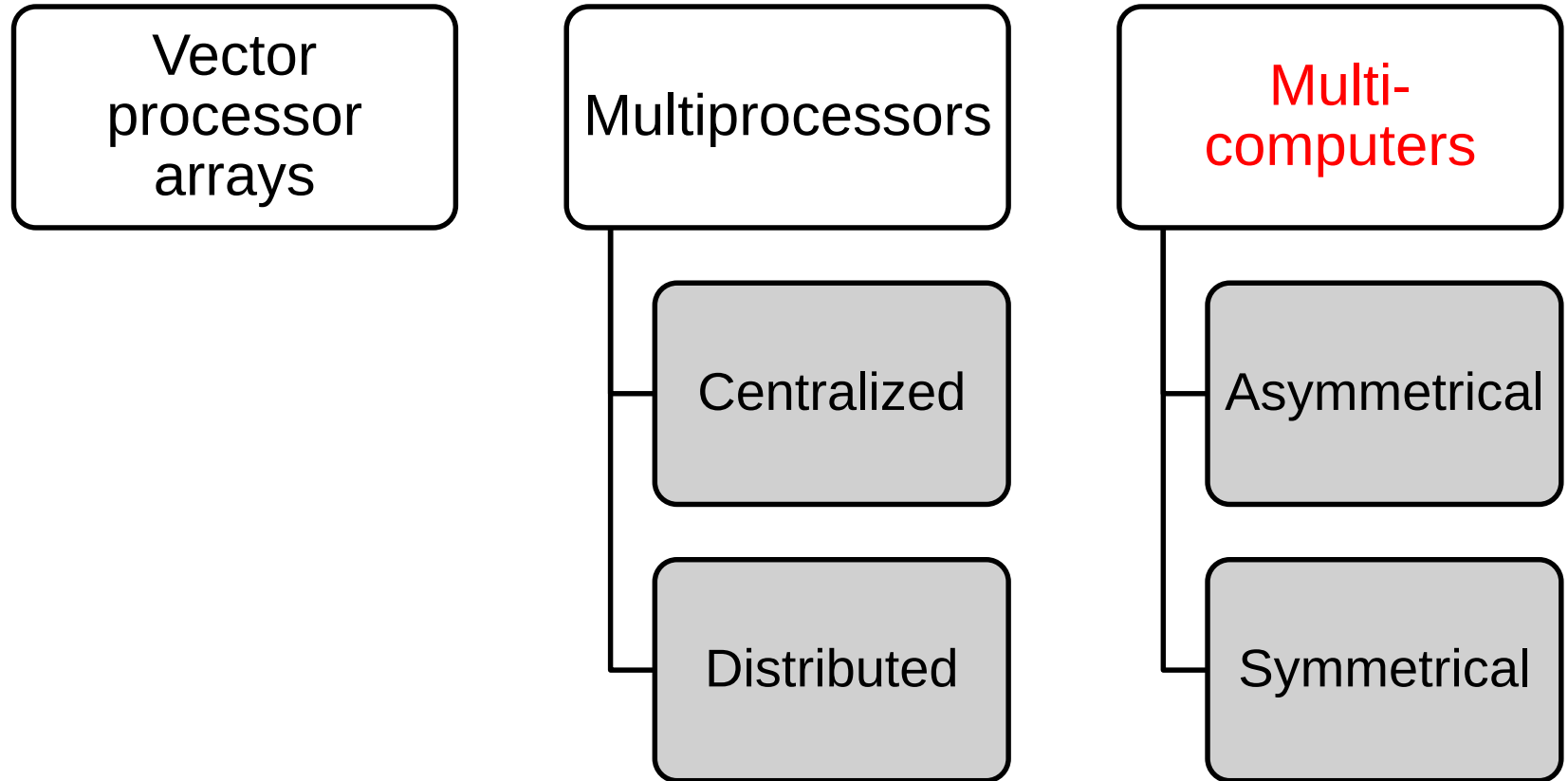
Directory-based Protocol



Directory-based Protocol



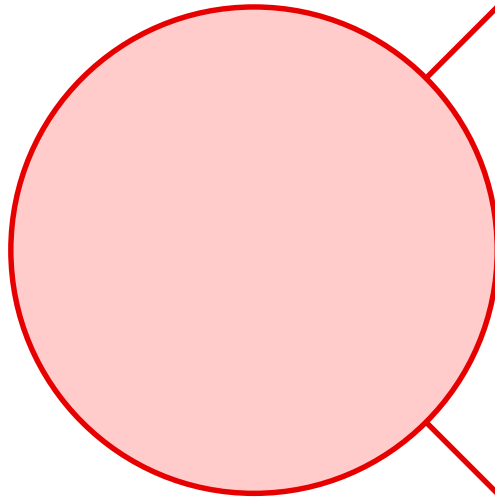
Parallel Architectures – 3 Popular ones



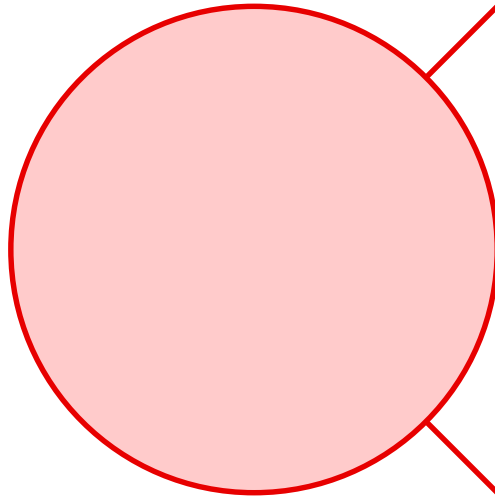
Multicomputers

- It is a multi-CPU distributed computer architecture.
- Such architecture has **disjoint local address spaces**.
 - i.e., there is no global address space as like in UMA or NUMA.
 - ie., Each processor has direct access to its own local memory.
- The processors interact with each other by passing messages. (MPI programming model).
- The switching network of such architectures should have
 - Low-latency
 - High bandwidth access (between processors)
- A good balance between the high speed processors and high speed networks is really a crucial task for multi-computers.

Types



**Asymmetrical
Multicomputers**



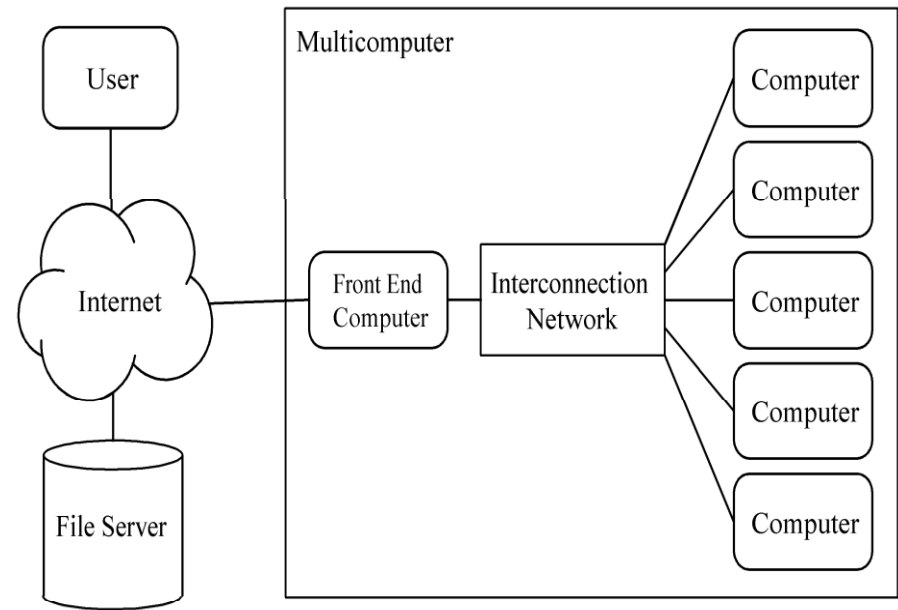
**Symmetrical
Multicomputers**

Asymmetric Multicomputers

- Early multicomputers had an asymmetric design.
- Here, there is a front-end that interacts with users and I/O devices.
 - Interacts with users and i/o devices
- Processors are placed in the back end.
 - Meant for parallel computations.
 - These backend processors run **primitive** operating system (specific OS which cannot be decomposed. E.g. VERTEX OS)
- Examples of asymmetrical multicomputers
 - Eg. Intel iPSC and nCUBE/ten

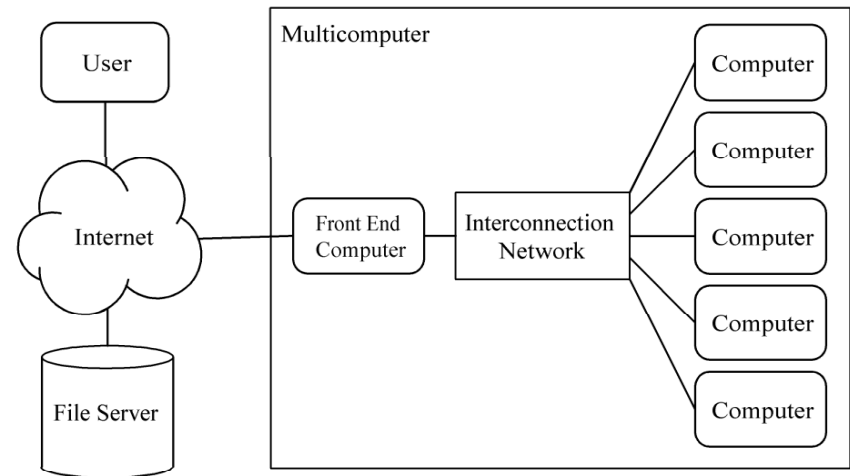
Advantages

- The back-end processors are dedicated to parallel computations.
 - Hence, they are easier to understand, model, or tune performance
- Only a simple back-end operating system is needed for all parallel computing nodes.
 - Thus, it is easy for a vendor to establish the units.



Disadvantages

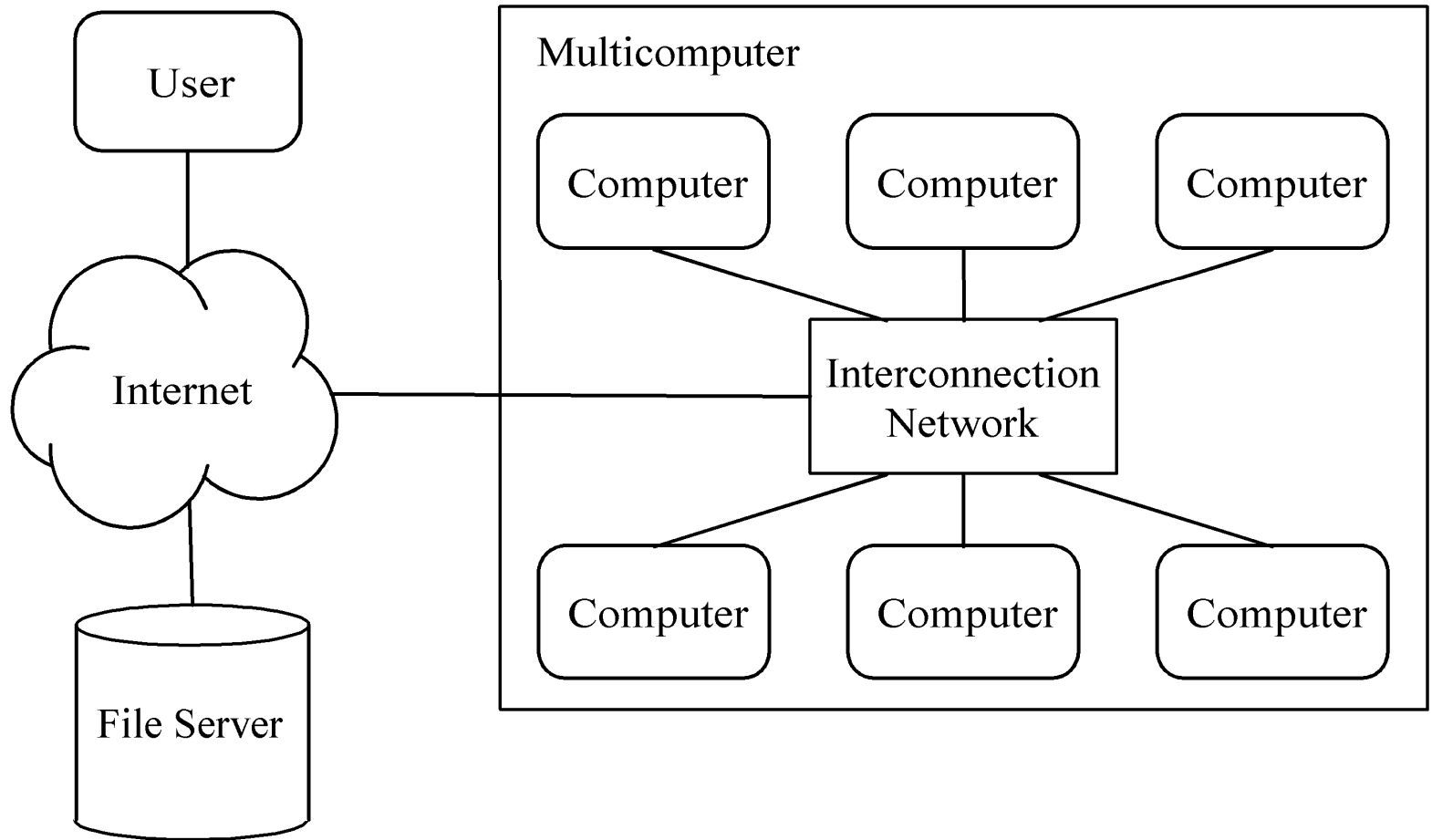
- Front-end computer is a single point of failure.
- Single front-end computer limits the scalability of system
- Primitive operating system in back-end processors makes debugging difficult
- Every application requires development of both front-end and back-end program



Symmetric Multicomputers

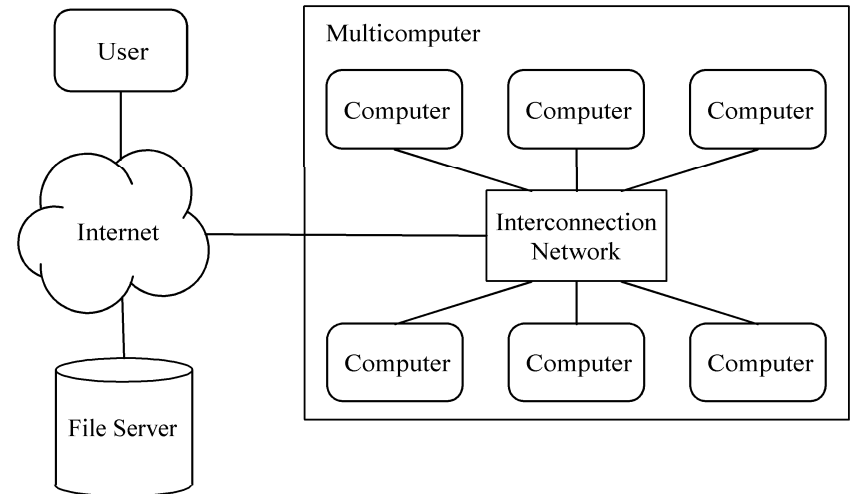
- Every computer executes **the same operating system** and has identical functionality.
- Users may log into any computer to edit or compile their programs.
- Any or all computers may be involved in the execution of their program.
- During execution of programs, every PE executes **the same program**.
- When only one PE should execute an operation, an *“if”* statement is used to select the PE.

Symmetric Multicomputers



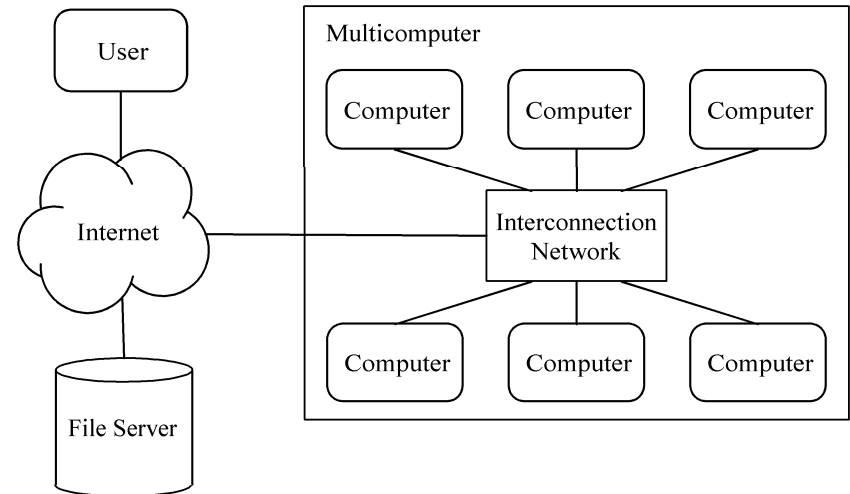
Advantages

- It alleviates performance bottleneck caused by single front-end computer
- Debugging is easier
- Every processor executes the same program.



Disadvantages of Sym. MultiComputers

- More difficult to maintain illusion of single “parallel computer”
- No simple way to balance program development workload among processors
- More difficult to achieve high performance when multiple processes run on each processor.



Multi-threading

- Hardware multi-threading allows multiple threads to share the functional units of a single processor.
 - Hardware-level (registers are duplicated).
- Each thread would have a separate copy of register file and the PC.
- The memory is also shared through the virtual memory mechanisms (to support multi-programming).
- 2 approaches
 - Fine-grained multi-threading
 - Here, the hardware suggests switching between threads after each instructions.
 - Coarse-grained multi-threading
 - Here, the hardware suggests switching between threads only if specific events occur (for e.g., cache misses).

Goals of parallel computing

- Reduce Wall Clock Time

- wall clock time is not the CPU time

- Cheapest Possible Solution Strategy

- many workstations cost much less than a large mainframe-style of implementation.

- Locality

- processors should be “close” to one another in terms of communication
 - the more communication latency incurred, the longer it will run.

- Memory Constraints

- As researcher becomes more sophisticated, the complexity of problems they tackle increases proportionally.
 - One of the first resources to get exhausted is local memory. Distributed memory helps alleviate the memory problem

Performance Metrics

- Execution Time

- The **serial runtime** of a program is the time elapsed between the beginning and the end of its execution on a sequential computer.
- The **parallel runtime** is the time that elapses from the moment a parallel computation starts to the moment the last processing element finishes execution. We denote the serial runtime by T_S and the parallel runtime by T_P .

- Total Parallel Overhead:

- **Total overhead** of a parallel system is defined as the total time collectively spent by all the processing elements over and above that required by the fastest known sequential algorithm for solving the same problem on a single processing element.
- i.e., $T_o = pT_P - T_S$.

Performance Metrics

- Speedup

- It is defined as the ratio of the time taken to solve a problem on a single processing element to the time required to solve the same problem on a parallel computer with p identical processing elements. We denote speedup by the symbol S .
- **speedup** is a number that measures the relative performance of two systems processing the same problem (Wikipedia)

- Efficiency

- **Efficiency** is a measure of the fraction of time for which a processing element is usefully employed;
- it is defined as the ratio of speedup to the number of processing elements.

$$\text{Speedup} = \frac{T_s(n)}{T_p(n, p)}$$
$$\text{Efficiency} = \frac{\text{Speedup}}{p} = \frac{T_s(n)}{p T_p(n, p)}$$

Performance Metrics

- Cost

- the **cost** of solving a problem on a parallel system is defined as the product of parallel runtime and the number of processing elements used.
- Cost reflects the sum of the time that each processing element spends solving the problem.

- Energy Consumption

- Energy consumed in joules while computing happens at PEs.
- It is dependent on the frequency of CPU utilized and the execution time.

Performance metrics

- MIPS
 - Million Instructions per Second
- MFLOPS
 - Millions of Floating Point operations per second
- Utilization
 - It is the ratio between the computation capacity used during parallel execution and the capacity that was available.